Hybrid kriging try

BOIKARABELO OLIFANT

2026-08-20

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## Loading required package: lattice
```

```
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.5.3
```

```
## Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

```
library(ggplot2)


airbnb <- readRDS("C:/Users/boika/Downloads/California_airbnb.csv/California_airbnb.rds")
#head(airbnb)

# Drop redundant textual indices and high-missing-ratio variables
airbnb1 <- unique(airbnb)
airbnb2 <- airbnb1[, setdiff(names(airbnb1), c("scrape_id", "host_id", "name", "host_country", "bedrooms
```

id_scape, name, host_country must be removed since they serve little purpose.

bedrooms column removed since it has too many missing values.

The "Date" columns next

```
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
# Breakpoint for restarts
bnb <- airbnb2

date_cols <- c("first_review", "last_review", "host_since", "calendar_last_scraped", "Date")

# Process, create numeric features, and drop original raw dimensions
for (col in date_cols) {
  new_num_name <- paste0(col, "_num")

  # Convert calendar timestamps to continuous day counts since Unix epoch
  bnb[[new_num_name]] <- as.numeric(ymd(bnb[[col]]))

  # Delete original text timestamp fields
  bnb[[col]] <- NULL
}

str(bnb$Date_num)
```

```
##  num [1:247859] 19701 19701 19701 19701 19701 ...
```

Prep for preprocessing after spliting.

handling the missing values in the review scores.

```r
score_cols <- c(
  "review_scores_rating", "review_scores_accuracy", "review_scores_cleanliness",
  "review_scores_checkin", "review_scores_communication", "review_scores_location",
  "review_scores_value", "reviews_per_month"
)

# Create missing listings tracker column indicator flag
bnb$is_new_listing <- as.integer(is.na(bnb$review_scores_rating))
```

```r
# Split data (70/30)

set.seed(1852)
train_idx  <- sample(1:nrow(bnb), 0.7 * nrow(bnb))
train_data <- bnb[train_idx, ]
test_data  <- bnb[-train_idx, ]

# Filter zero/near-zero variance predictors across the partitions
remove_tcols <- nearZeroVar(train_data, names = TRUE)

train_data <- train_data[, setdiff(names(train_data), remove_tcols)]
test_data  <- test_data[, setdiff(names(test_data), remove_tcols)]

# Impute evaluation attributes with median metrics calculated strictly on training observations
for (col in score_cols) {
  train_median <- median(train_data[[col]], na.rm = TRUE)

  train_data[[col]][is.na(train_data[[col]])] <- train_median
```

```
  test_data[[col]][is.na(test_data[[col]])] <- train_median
}

head(train_data)
```

```
##                 id host_response_time host_acceptance_rate host_is_superhost
## 14535  3.908285e+07                                    NA                 0
## 180050 3.230752e+07 a few days or more                  33                NA
## 69962  8.666864e+06 within a few hours                  55                 1
## 192041 2.961060e+07     within an hour                 100                 1
## 228008 2.132914e+07     within an hour                 100                 0
## 186526 7.607613e+17     within an hour                 100                 0
##        host_listings_count host_total_listings_count host_identity_verified
## 14535                    2                         3                      1
## 180050                   3                         3                      0
## 69962                    1                         2                      1
## 192041                   7                         7                      0
## 228008                  13                        27                      1
## 186526                   3                         3                      0
##        neighbourhood_cleansed latitude longitude        property_type
## 14535           Redondo Beach 33.83633 -118.3880   Entire rental unit
## 180050        University City 32.84812 -117.2035 Private room in home
## 69962               San Bruno 37.63295 -122.4488   Entire guest suite
## 192041               La Jolla 32.84355 -117.2678          Entire home
## 228008    South San Francisco 37.66486 -122.4021 Private room in home
## 186526               Southwest 32.61261 -117.0585          Entire home
##             room_type accommodates beds price minimum_nights maximum_nights
## 14535  Entire home/apt            1    1   110             90            360
## 180050    Private room            1    1   125             31           1125
## 69962  Entire home/apt            4    1   135              4           1125
## 192041 Entire home/apt            6    3   604              4             60
## 228008    Private room            2    1    55              1             28
## 186526 Entire home/apt           16    8   314             30            365
##        minimum_minimum_nights maximum_minimum_nights minimum_maximum_nights
## 14535                      90                    90                   360
## 180050                     31                    31                  1125
## 69962                       4                     4                  1125
## 192041                      4                     5                  1125
## 228008                      1                     1                    28
## 186526                     30                    30                   365
##        maximum_maximum_nights minimum_nights_avg_ntm maximum_nights_avg_ntm
## 14535                     360                    90                   360
## 180050                   1125                    31                  1125
## 69962                    1125                     4                  1125
## 192041                   1125                     4                  1125
## 228008                     28                     1                    28
## 186526                    365                    30                   365
##        has_availability availability_30 availability_60 availability_90
## 14535                 0               0              0               0
## 180050                1              27             57              87
## 69962                 1               0             19              49
## 192041                1              14             39              53
## 228008                1              25             55              85
```

```
## 186526               1               9              21              51
##        availability_365 number_of_reviews number_of_reviews_ltm
## 14535                 0                 2                     0
## 180050              362                 3                     0
## 69962               324                70                     2
## 192041              277               157                    32
## 228008              175               205                    58
## 186526              105                 6                     6
##        number_of_reviews_l30d review_scores_rating review_scores_accuracy
## 14535                       0                5.00                    5.00
## 180050                      0                4.67                    5.00
## 69962                       0                4.86                    4.87
## 192041                      0                4.92                    4.91
## 228008                      1                4.52                    4.69
## 186526                      1                4.33                    4.00
##        review_scores_cleanliness review_scores_checkin
## 14535                       5.00                 5.00
## 180050                      4.67                 5.00
## 69962                       4.84                 4.96
## 192041                      4.89                 4.97
## 228008                      4.39                 4.86
## 186526                      3.83                 5.00
##        review_scores_communication review_scores_location review_scores_value
## 14535                         5.00                   5.00                5.00
## 180050                        5.00                   5.00                5.00
## 69962                         4.96                   4.86                4.89
## 192041                        4.94                   4.94                4.84
## 228008                        4.76                   4.54                4.50
## 186526                        4.83                   4.17                4.50
##        instant_bookable calculated_host_listings_count
## 14535                 0                              2
## 180050                0                              3
## 69962                 0                              1
## 192041                1                              4
## 228008                0                             12
## 186526                0                              2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
## 180050                                           0
## 69962                                            1
## 192041                                           4
## 228008                                           0
## 186526                                           2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
##        bathrooms_text
## 14535   1 shared bath
## 180050 1 private bath
## 69962          1 bath
```

```
## 192041      3.5 baths
## 228008      1.5 baths
## 186526        2 baths
##
## 14535
## 180050
## 69962
## 192041
## 228008
## 186526 ["Carbon monoxide alarm", "Iron", "Stainless steel gas stove", "Freezer", "City skyline view"
##            host_city             City first_review_num last_review_num
## 14535  Redondo Beach      los-angeles            18217           18231
## 180050     San Diego        san-diego            18586           19137
## 69962      San Bruno san-mateo-county            16906           19602
## 192041     San Diego        san-diego            17869           19585
## 228008     Daly City san-mateo-county            17460           19590
## 186526                      san-diego            19322           19517
##        host_since_num calendar_last_scraped_num Date_num is_new_listing
## 14535           15961                     19695    19694              0
## 180050          17870                     19533    19532              0
## 69962           16340                     19713    19789              0
## 192041          15537                     19618    19618              0
## 228008          17053                     19618    19618              0
## 186526          18054                     19532    19532              0
```

```
head(train_data$bathrooms_text)
```

```
## [1] "1 shared bath"  "1 private bath" "1 bath"         "3.5 baths"
## [5] "1.5 baths"      "2 baths"
```

```
head(train_data)
```

```
##                  id host_response_time host_acceptance_rate host_is_superhost
## 14535  3.908285e+07                                     NA                 0
## 180050 3.230752e+07 a few days or more                   33                NA
## 69962  8.666864e+06 within a few hours                   55                 1
## 192041 2.961060e+07     within an hour                  100                 1
## 228008 2.132914e+07     within an hour                  100                 0
## 186526 7.607613e+17     within an hour                  100                 0
##        host_listings_count host_total_listings_count host_identity_verified
## 14535                    2                         3                     1
## 180050                   3                         3                     0
## 69962                    1                         2                     1
## 192041                   7                         7                     0
## 228008                  13                        27                     1
## 186526                   3                         3                     0
##        neighbourhood_cleansed latitude longitude        property_type
## 14535           Redondo Beach 33.83633 -118.3880   Entire rental unit
## 180050        University City 32.84812 -117.2035 Private room in home
## 69962               San Bruno 37.63295 -122.4488   Entire guest suite
## 192041               La Jolla 32.84355 -117.2678          Entire home
## 228008    South San Francisco 37.66486 -122.4021 Private room in home
## 186526              Southwest 32.61261 -117.0585          Entire home
```

```
##             room_type accommodates beds price minimum_nights maximum_nights
## 14535  Entire home/apt            1    1   110             90            360
## 180050    Private room            1    1   125             31           1125
## 69962  Entire home/apt            4    1   135              4           1125
## 192041 Entire home/apt            6    3   604              4             60
## 228008    Private room            2    1    55              1             28
## 186526 Entire home/apt           16    8   314             30            365
##        minimum_minimum_nights maximum_minimum_nights minimum_maximum_nights
## 14535                      90                    90                   360
## 180050                     31                    31                  1125
## 69962                       4                     4                  1125
## 192041                      4                     5                  1125
## 228008                      1                     1                    28
## 186526                     30                    30                   365
##        maximum_maximum_nights minimum_nights_avg_ntm maximum_nights_avg_ntm
## 14535                     360                    90                   360
## 180050                   1125                    31                  1125
## 69962                    1125                     4                  1125
## 192041                   1125                     4                  1125
## 228008                     28                     1                    28
## 186526                    365                    30                   365
##        has_availability availability_30 availability_60 availability_90
## 14535                 0               0              0              0
## 180050                1              27             57             87
## 69962                 1               0             19             49
## 192041                1              14             39             53
## 228008                1              25             55             85
## 186526                1               9             21             51
##        availability_365 number_of_reviews number_of_reviews_ltm
## 14535                 0                 2                    0
## 180050              362                 3                    0
## 69962               324                70                    2
## 192041              277               157                   32
## 228008              175               205                   58
## 186526              105                 6                    6
##        number_of_reviews_l30d review_scores_rating review_scores_accuracy
## 14535                       0                5.00                   5.00
## 180050                      0                4.67                   5.00
## 69962                       0                4.86                   4.87
## 192041                      0                4.92                   4.91
## 228008                      1                4.52                   4.69
## 186526                      1                4.33                   4.00
##        review_scores_cleanliness review_scores_checkin
## 14535                       5.00                 5.00
## 180050                      4.67                 5.00
## 69962                       4.84                 4.96
## 192041                      4.89                 4.97
## 228008                      4.39                 4.86
## 186526                      3.83                 5.00
##        review_scores_communication review_scores_location review_scores_value
## 14535                         5.00                   5.00                5.00
## 180050                        5.00                   5.00                5.00
## 69962                         4.96                   4.86                4.89
## 192041                        4.94                   4.94                4.84
```

```
## 228008                       4.76                   4.54                4.50
## 186526                       4.83                   4.17                4.50
##        instant_bookable calculated_host_listings_count
## 14535                 0                               2
## 180050                0                               3
## 69962                 0                               1
## 192041                1                               4
## 228008                0                              12
## 186526                0                               2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
## 180050                                           0
## 69962                                            1
## 192041                                           4
## 228008                                           0
## 186526                                           2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
##        bathrooms_text
## 14535   1 shared bath
## 180050 1 private bath
## 69962          1 bath
## 192041      3.5 baths
## 228008      1.5 baths
## 186526        2 baths
##
## 14535
## 180050
## 69962
## 192041
## 228008
## 186526 ["Carbon monoxide alarm", "Iron", "Stainless steel gas stove", "Freezer", "City skyline view"
##            host_city             City first_review_num last_review_num
## 14535  Redondo Beach      los-angeles            18217           18231
## 180050     San Diego        san-diego            18586           19137
## 69962      San Bruno san-mateo-county            16906           19602
## 192041     San Diego        san-diego            17869           19585
## 228008     Daly City san-mateo-county            17460           19590
## 186526                      san-diego            19322           19517
##        host_since_num calendar_last_scraped_num Date_num is_new_listing
## 14535           15961                     19695    19694              0
## 180050          17870                     19533    19532              0
## 69962           16340                     19713    19789              0
## 192041          15537                     19618    19618              0
## 228008          17053                     19618    19618              0
## 186526          18054                     19532    19532              0
```

```
head(train_data$bathrooms_text)
```

```
## [1] "1 shared bath"  "1 private bath" "1 bath"         "3.5 baths"
## [5] "1.5 baths"      "2 baths"
```

```
sum(train_data$host_is_superhost)/length(train_data$host_is_superhost)
```

```
## [1] NA
```

majority class is 0

Thus we shall impute by mode

```
sum(train_data$host_is_superhost) / length(train_data$host_is_superhost)
```

```
## [1] NA
```

```
# Impute categorical superhost attributes using the dominant baseline majority mode (0)
train_data$host_is_superhost[is.na(train_data$host_is_superhost)] <- 0
test_data$host_is_superhost[is.na(test_data$host_is_superhost)]   <- 0

# Scale host acceptance metrics down to standard ratio probabilities
train_data$host_acceptance_rate <- train_data$host_acceptance_rate / 100
test_data$host_acceptance_rate  <- test_data$host_acceptance_rate / 100

# Set negative flags for tree architectures to safely isolate missing metrics natively
train_data$host_acceptance_rate[is.na(train_data$host_acceptance_rate)] <- -1
test_data$host_acceptance_rate[is.na(test_data$host_acceptance_rate)]   <- -1
```

```
# Scale host acceptance metrics down to standard ratio probabilities
train_data$host_acceptance_rate <- train_data$host_acceptance_rate / 100
test_data$host_acceptance_rate  <- test_data$host_acceptance_rate / 100

# Set negative flags for tree architectures to safely isolate missing metrics natively
train_data$host_acceptance_rate[is.na(train_data$host_acceptance_rate)] <- -1
test_data$host_acceptance_rate[is.na(test_data$host_acceptance_rate)]   <- -1
```

one-hot encoding room_type column

```
# Compute contrast dummy rules dropping primary reference fields (FullRank = TRUE)
encoder_rules <- dummyVars(formula = ~ room_type, data = train_data, fullRank = TRUE)

train_encoded <- as.data.frame(predict(encoder_rules, newdata = train_data))
test_encoded  <- as.data.frame(predict(encoder_rules, newdata = test_data))

train_data <- cbind(train_data, train_encoded)
test_data  <- cbind(test_data, test_encoded)

train_data$room_type <- NULL
test_data$room_type  <- NULL
```

```
# Re-run full-rank contrast definitions across regional geographical attributes
city_encoder <- dummyVars(formula = ~ City, data = train_data, fullRank = TRUE)

train_city_encoded <- as.data.frame(predict(city_encoder, newdata = train_data))
test_city_encoded  <- as.data.frame(predict(city_encoder, newdata = test_data))

train_data <- cbind(train_data, train_city_encoded)
test_data  <- cbind(test_data, test_city_encoded)

train_data$City <- NULL
test_data$City  <- NULL
```

One-hot encoding on the amenities column.

```
library(stringr)

target_amenities <- c(
  "smoke alarm", "toaster", "mini fridge", "hot water",
  "dishwasher", "Air conditioning", "Kitchen", "wifi", " Refrigerator", "Hot water", "Microwave "
)

engineer_amenities <- function(df) {
  df$amenities_clean <- tolower(df$amenities)
  df$amenities_clean <- str_replace_all(df$amenities_clean, pattern = c("\\[" = "", "\\]" = "", "\\\\" =

  for (amenity in target_amenities) {
    col_name <- paste0("has_", gsub(" ", "_", amenity))
    df[[col_name]] <- ifelse(str_detect(df$amenities_clean, fixed(amenity)), 1, 0)
  }

  df$amenities       <- NULL
  df$amenities_clean <- NULL
  return(df)
}

train_data <- engineer_amenities(train_data)
test_data  <- engineer_amenities(test_data)
```

```
# Process categorical continuous communication channels
encode_response_time <- function(column) {
  char_col <- trimws(as.character(column))
  encoded  <- rep(0, length(char_col))

  encoded[char_col == "within an hour"]        <- 4
  encoded[char_col == "within a few hours"]    <- 3
  encoded[char_col == "within a day"]          <- 2
  encoded[char_col == "a few days or more"]    <- 1

  encoded[char_col %in% c("N/A", "unknown", "blank", "")] <- 0
  encoded[is.na(char_col)]                                <- 0
  return(as.integer(encoded))
```

```
}

train_data$host_response_time_num <- encode_response_time(train_data$host_response_time)
test_data$host_response_time_num  <- encode_response_time(test_data$host_response_time)

train_data$host_response_time <- NULL
test_data$host_response_time  <- NULL
```

The bathroom_text must be converted to numeric

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(readr)

num_bathrooms <- function(df) {
  df %>%
    mutate(
      bath_temp = tolower(trimws(bathrooms_text)),
      bath_temp = case_when(
        grepl("half-bath", bath_temp) ~ "0.5",
        bath_temp == "shared baths"   ~ "0",
        TRUE                          ~ bath_temp
      ),
      bathrooms_numeric = parse_number(bath_temp)
    ) %>%
    select(-bath_temp, -bathrooms_text)
}

train_cleaned <- num_bathrooms(train_data)
test_cleaned  <- num_bathrooms(test_data)
```

```
# Evacuate standard incomplete rows
train_cleaned <- na.omit(train_cleaned)
test_cleaned  <- na.omit(test_cleaned)

# Convert the pricing distributions to log scales
train_cleaned$logprice <- log(train_cleaned$price)
test_cleaned$logprice  <- log(test_cleaned$price)
```

```r
# Compute absolute IQR metrics using the training baseline subset
iqr <- IQR(train_cleaned$logprice)
Q   <- quantile(train_cleaned$logprice, probs = c(.25, .75), na.rm = FALSE)
U   <- Q[2] + 1.5 * iqr
D   <- Q[1] - 1.5 * iqr

# Using log scale to trim the data.
train_data1 <- subset(train_cleaned, logprice > D & logprice < 8)
test_data1  <- subset(test_cleaned, logprice > D & logprice < 8)

#head(train_data1)
```

```r
pdf("distribution_raw_price.pdf", width = 6, height = 5)
hist(train_cleaned$price,
     main = "Distribution of Raw Airbnb Prices",
     xlab = "Price ($)",
     col = "lightblue",
     border = "white")
dev.off()
```

```
## pdf
##   2
```

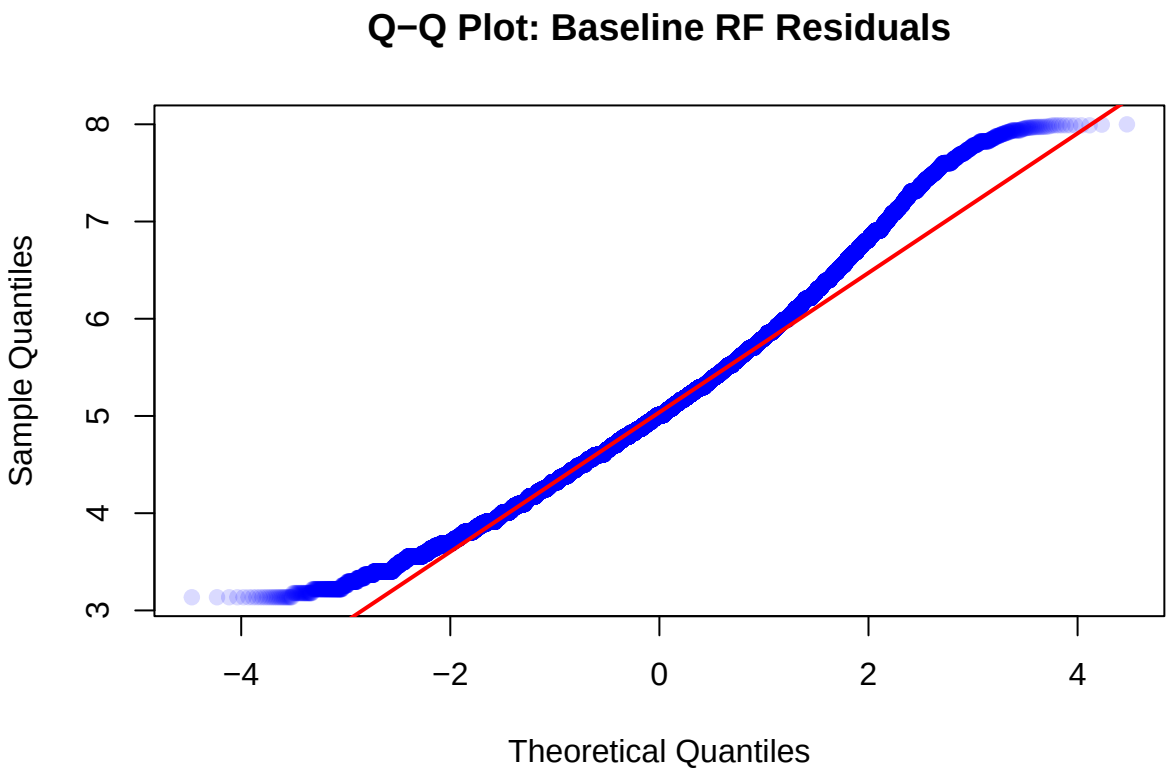
```r
pdf("distribution_filtered_log_price.pdf", width = 6, height = 5)
hist(train_data1$logprice,
     main = "Distribution of Filtered Log Prices",
     xlab = "Log(Price)",
     col = "lightgreen",
     border = "white")
dev.off()
```

```
## pdf
##   2
```

```r
pdf("distribution_filtered_raw_price.pdf", width = 6, height = 5)
hist(train_data1$price,
     main = "Distribution of Filtered Raw Prices",
     xlab = "Price ($)",
     col = "coral",
     border = "white")
dev.off()
```

```
## pdf
##   2
```

```r
qqnorm(train_data1$logprice, main = "Q-Q Plot: Baseline RF Residuals", col = rgb(0,0,1, alpha=0.15), pc
qqline(train_data1$logprice, col = "red", lwd = 2)
```



Random forest model will be fitted thus, first the non-numeric variables are removed

Then, longitude and latitude, for proper application of the hybrid method.

```
train_final <- train_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
          -latitude, -longitude,-host_city, -logprice)

test_final <- test_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
          -latitude, -longitude,-host_city, -logprice)
```

1. Grid search on the ranger fit random forest.

```
library(ranger)


set.seed(1852)
sub_sample_size <- min(5000, nrow(train_final))


train_sub <- train_final[sample(1:nrow(train_final), sub_sample_size),]
```

```r
# Define a light grid
hyper_grid <- expand.grid(
  mtry          = c(2, 4, 8, 22),
  min.node.size = c(5, 10, 20),
  oob_rmse      = 0
)


for(i in 1:nrow(hyper_grid)) {

  model <- ranger(
    dependent.variable.name = "price",
    data            = train_sub,
    num.trees       = 150,
    mtry            = hyper_grid$mtry[i],
    min.node.size   = hyper_grid$min.node.size[i],
    max.depth       = 12,
    save.memory     = TRUE,
    num.threads     = 2,
    seed            = 1893
  )

  hyper_grid$oob_rmse[i] <- sqrt(model$prediction.error)

  rm(model)
  gc()
}

print(hyper_grid)
```

```
##    mtry min.node.size oob_rmse
## 1     2             5 202.5373
## 2     4             5 187.8545
## 3     8             5 181.1444
## 4    22             5 178.5025
## 5     2            10 203.7889
## 6     4            10 187.9468
## 7     8            10 180.3421
## 8    22            10 178.7391
## 9     2            20 203.3093
## 10    4            20 188.6884
## 11    8            20 181.0794
## 12   22            20 178.7109
```

Chosen RF model from the above grid search has mtry of 8 and min.node.size of 5, this is to avoid overfitting

```r
set.seed(1852)

rf1 <- ranger(
```

```
  dependent.variable.name = "price",
  data                    = train_final,
  num.trees               = 100,
  importance              = "permutation",
  mtry                    = 8,
  min.node.size           = 5,
  save.memory             = TRUE
)
```

```
## Growing trees.. Progress: 35%. Estimated remaining time: 57 seconds.
## Growing trees.. Progress: 73%. Estimated remaining time: 23 seconds.
## Computing permutation importance.. Progress: 29%. Estimated remaining time: 1 minute, 15 seconds.
## Computing permutation importance.. Progress: 57%. Estimated remaining time: 48 seconds.
## Computing permutation importance.. Progress: 85%. Estimated remaining time: 16 seconds.
```

```
print(rf1)
```

```
## Ranger result
##
## Call:
##  ranger(dependent.variable.name = "price", data = train_final,      num.trees = 100, importance = "p
##
## Type:                             Regression
## Number of trees:                  100
## Sample size:                      129120
## Number of independent variables:  64
## Mtry:                             8
## Target node size:                 5
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       20818.64
## R squared (OOB):                  0.6544833
```

Drop useless variables that scored 0 or negative in your importance test

```
rf_importance <- importance(rf1)
importance_df <- data.frame(
  Feature    = names(rf_importance),
  Importance = as.numeric(rf_importance)
)



# Sort features descending by predictive value
importance_df   <- importance_df[order(-importance_df$Importance), ]
sorted_features <- importance_df$Feature



importance_threshold <- 1000
```

```r
# Automatically identify features below or equal to the threshold
useless_features_auto <- importance_df$Feature[importance_df$Importance <= importance_threshold]

# Print out the automatically caught features for transparency
cat("--- Automatically Dropping", length(useless_features_auto), "Useless Features ---\n")
```

```
## --- Automatically Dropping 27 Useless Features ---
```

```r
print(useless_features_auto)
```

```
##  [1] "Citysan-francisco"         "Date_num"
##  [3] "calendar_last_scraped_num" "host_is_superhost"
##  [5] "instant_bookable"          "Citysanta-clara-county"
##  [7] "room_typeShared room"      "Cityoakland"
##  [9] "Citysanta-cruz-county"     "host_identity_verified"
## [11] "Citysan-mateo-county"      "has_wifi"
## [13] "has_availability"          "Citypacific-grove"
## [15] "has_smoke_alarm"           "bathrooms_numeric"
## [17] "has_toaster"               "has_hot_water"
## [19] "has_mini_fridge"           "has_dishwasher"
## [21] "room_typeHotel room"       "is_new_listing"
## [23] "has_Air_conditioning"      "has_Kitchen"
## [25] "has__Refrigerator"         "has_Hot_water"
## [27] "has_Microwave_"
```

```r
keep_features <- setdiff(names(train_final), useless_features_auto)

train_final <- train_final[, keep_features]
test_final  <- test_final[, keep_features]


# Verification Check
cat("\nRemaining column count in training set:", ncol(train_final), "\n")
```

```
##
## Remaining column count in training set: 38
```

```r
head(train_final)
```

```
##        host_acceptance_rate host_listings_count host_total_listings_count
## 14535               -0.0100                   2                         3
## 180050               0.0033                   3                         3
## 69962                0.0055                   1                         2
## 192041               0.0100                   7                         7
## 228008               0.0100                  13                        27
## 186526               0.0100                   3                         3
##        accommodates beds price minimum_nights maximum_nights
## 14535             1    1   110             90            360
## 180050            1    1   125             31           1125
## 69962             4    1   135              4           1125
```

```
## 192041            6    3   604              4             60
## 228008            2    1    55              1             28
## 186526           16    8   314             30            365
##        minimum_minimum_nights maximum_minimum_nights minimum_maximum_nights
## 14535                      90                     90                    360
## 180050                     31                     31                   1125
## 69962                       4                      4                   1125
## 192041                      4                      5                   1125
## 228008                      1                      1                     28
## 186526                     30                     30                    365
##        maximum_maximum_nights minimum_nights_avg_ntm maximum_nights_avg_ntm
## 14535                     360                     90                    360
## 180050                   1125                     31                   1125
## 69962                    1125                      4                   1125
## 192041                   1125                      4                   1125
## 228008                     28                      1                     28
## 186526                    365                     30                    365
##        availability_30 availability_60 availability_90 availability_365
## 14535                0               0               0                0
## 180050              27              57              87              362
## 69962                0              19              49              324
## 192041              14              39              53              277
## 228008              25              55              85              175
## 186526               9              21              51              105
##        number_of_reviews number_of_reviews_ltm number_of_reviews_l30d
## 14535                  2                     0                     0
## 180050                 3                     0                     0
## 69962                 70                     2                     0
## 192041               157                    32                     0
## 228008               205                    58                     1
## 186526                 6                     6                     1
##        review_scores_rating review_scores_accuracy review_scores_cleanliness
## 14535                  5.00                   5.00                    5.00
## 180050                 4.67                   5.00                    4.67
## 69962                  4.86                   4.87                    4.84
## 192041                 4.92                   4.91                    4.89
## 228008                 4.52                   4.69                    4.39
## 186526                 4.33                   4.00                    3.83
##        review_scores_checkin review_scores_communication review_scores_location
## 14535                   5.00                        5.00                   5.00
## 180050                  5.00                        5.00                   5.00
## 69962                   4.96                        4.96                   4.86
## 192041                  4.97                        4.94                   4.94
## 228008                  4.86                        4.76                   4.54
## 186526                  5.00                        4.83                   4.17
##        review_scores_value calculated_host_listings_count
## 14535                 5.00                              2
## 180050                5.00                              3
## 69962                 4.89                              1
## 192041                4.84                              4
## 228008                4.50                             12
## 186526                4.50                              2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
```

```
## 180050                                          0
## 69962                                           1
## 192041                                          4
## 228008                                          0
## 186526                                          2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
##        first_review_num last_review_num host_since_num room_typePrivate room
## 14535             18217           18231          15961                      0
## 180050            18586           19137          17870                      1
## 69962             16906           19602          16340                      0
## 192041            17869           19585          15537                      0
## 228008            17460           19590          17053                      1
## 186526            19322           19517          18054                      0
##        Citysan-diego host_response_time_num
## 14535              0                     0
## 180050             1                     1
## 69962              0                     3
## 192041             1                     4
## 228008             0                     4
## 186526             1                     4
```

```
rf2 <- ranger(
  dependent.variable.name = "price",
  data                    = train_final,
  num.trees               = 100,
  importance              = "permutation",
  mtry                    = 8,
  min.node.size           = 5,
  save.memory             = TRUE
)
```

```
## Growing trees.. Progress: 29%. Estimated remaining time: 1 minute, 18 seconds.
## Growing trees.. Progress: 58%. Estimated remaining time: 45 seconds.
## Growing trees.. Progress: 87%. Estimated remaining time: 14 seconds.
## Computing permutation importance.. Progress: 44%. Estimated remaining time: 39 seconds.
## Computing permutation importance.. Progress: 96%. Estimated remaining time: 2 seconds.
```

```
print(rf2)
```

```
## Ranger result
##
## Call:
##  ranger(dependent.variable.name = "price", data = train_final,      num.trees = 100, importance = "pe
##
## Type:                             Regression
## Number of trees:                  100
## Sample size:                      129120
```

```
## Number of independent variables:  37
## Mtry:                              8
## Target node size:                  5
## Variable importance mode:          permutation
## Splitrule:                         variance
## OOB prediction error (MSE):        19103.02
## R squared (OOB):                   0.6829566
```

```
importance(rf2)
```

```
##                          host_acceptance_rate
##                                     3581.5443
##                           host_listings_count
##                                     5496.3672
##                     host_total_listings_count
##                                     6465.5570
##                                  accommodates
##                                    38328.2639
##                                          beds
##                                    14849.6822
##                                minimum_nights
##                                     8443.0384
##                                maximum_nights
##                                     2102.8384
##                           minimum_minimum_nights
##                                     9223.6131
##                           maximum_minimum_nights
##                                     7520.4369
##                           minimum_maximum_nights
##                                     1740.8437
##                           maximum_maximum_nights
##                                     1557.5883
##                           minimum_nights_avg_ntm
##                                    11643.7010
##                           maximum_nights_avg_ntm
##                                     1739.3186
##                                availability_30
##                                     5129.4356
##                                availability_60
##                                     4515.7628
##                                availability_90
##                                     4298.7192
##                               availability_365
##                                     4547.4440
##                               number_of_reviews
##                                     5897.5693
##                            number_of_reviews_ltm
##                                     5621.0576
##                           number_of_reviews_l30d
##                                     1691.0527
##                             review_scores_rating
##                                     7869.4034
##                           review_scores_accuracy
##                                     3510.2602
```

```
##                     review_scores_cleanliness
##                                       4507.9323
##                         review_scores_checkin
##                                       2450.3472
##                   review_scores_communication
##                                       2459.2923
##                        review_scores_location
##                                      10750.9957
##                          review_scores_value
##                                       4649.3644
##               calculated_host_listings_count
##                                       4092.2562
##  calculated_host_listings_count_entire_homes
##                                       6765.2866
## calculated_host_listings_count_private_rooms
##                                       5037.2544
##                             reviews_per_month
##                                      14399.8383
##                              first_review_num
##                                       8980.4421
##                               last_review_num
##                                       9381.9422
##                                host_since_num
##                                       4887.8790
##                          room_typePrivate room
##                                       4889.2861
##                                  Citysan-diego
##                                        718.4704
##                         host_response_time_num
##                                       1960.2263
```

1. Predictions & residuals for the training and test sets.

```
train_data1$y_pred <- predict(rf2, data = train_final)$predictions
train_data1$rf_residuals <- train_data1$price - train_data1$y_pred
```

Residual check for normality

```
pdf("RF_Residual_Normality_Diagnostics.pdf", width = 10, height = 8)

# Set up a 2x2 grid framework for side-by-side visual analysis
par(mfrow = c(1, 2))


# Extract baseline training residuals (used as the input for Kriging)
rf_res <- train_data1$rf_residuals

# Plot 1: Histogram of Baseline RF Residuals
hist(rf_res, col = "skyblue",xlim=c(-700, 700), main = "Histogram: Baseline RF Residuals",
     xlab = "Residual Error ($)", border = "white")
```

```r
abline(v = mean(rf_res, na.rm=TRUE), col = "red", lwd = 2, lty = 2)

# Plot 2: Normal Q-Q Plot of Baseline RF Residuals
qqnorm(rf_res, main = "Q-Q Plot: Baseline RF Residuals", col = rgb(0,0,1, alpha=0.15), pch=19)
qqline(rf_res, col = "red", lwd = 2)


# Reset plotting layout parameters
par(mfrow = c(1, 1))
dev.off()
```

```
## pdf
##   2
```

2. Spatial conversion

```r
library(sf)
library(gstat)
```

```
## Warning: package 'gstat' was built under R version 4.5.3
```

```r
library(ggplot2)

# Convert flat data to spatial coordinates using standard WGS84
spatial_train_geo <- st_as_sf(train_data1, coords = c("longitude", "latitude"), crs = 4326)
spatial_test_geo  <- st_as_sf(test_data1, coords = c("longitude", "latitude"), crs = 4326)

# NAD83: 3310 ( crs for California) covers the whole state
spatial_train_projected <- st_transform(spatial_train_geo, crs = 3310)
spatial_test_projected  <- st_transform(spatial_test_geo, crs = 3310)

# Jitter coordinate duplicates to avoid spatial distance errors (0-meter distance bugs)
spatial_train <- st_jitter(spatial_train_projected, amount = 1.0)
spatial_test  <- st_jitter(spatial_test_projected, amount = 1.0)


# Check training residuals
any(is.na(spatial_train$rf_residuals))
```

```
## [1] FALSE
```

```r
# Missing value check
any(is.na(spatial_test$price))
```

```
## [1] FALSE
```

```r
any(is.na(spatial_test$rf_residuals))
```

```
## [1] FALSE
```

1. VARIOGRAM computation and stable fitting of the kriging model on residuals

```
sub_sample_size <- min(5000, nrow(spatial_train))
spatial_train_sub <- spatial_train[sample(1:nrow(spatial_train), sub_sample_size), ]

# Compute empirical variogram
emp_vgm <- variogram(rf_residuals ~ 1, data = spatial_train_sub)

# Find total sample variance
init_variance <- var(spatial_train_sub$rf_residuals, na.rm = TRUE)

# Distributed variance stably between psill (70%) and nugget (30%)

init_vgm <- vgm(
  psill  = init_variance * 0.7,
  model  = "Sph",
  range  = 5000,
  nugget = init_variance * 0.3
)

# automated curve fitting
fitted_vgm <- fit.variogram(emp_vgm, model = init_vgm)
```

```
## Warning in fit.variogram(emp_vgm, model = init_vgm): singular model in
## variogram fit
```

```
print(fitted_vgm)
```

```
##   model    psill range
## 1   Nug 1098.425     0
## 2   Sph 2562.992  5000
```

```
# 2.Variogram plot
pdf("RF_residual.pdf", width = 6, height = 5 )
plot(
  emp_vgm,
  model = fitted_vgm,
  main  = list(label = "Fitted Semivariogram Model of Price RF Residuals", cex = 1.2),
  xlab  = "Distance (metres)",
  ylab  = "Semivariance"
)
dev.off()
```

```
## pdf
##   2
```

5.Test set prediction (Local Search Neighborhood used)

```r
test_rf_pred <- predict(rf2, data = test_final)$predictions

# Store the baseline prediction
test_data1$rf_pred <- test_rf_pred

# Verification checks
any(is.na(test_rf_pred))
```

```
## [1] FALSE
```

```r
summary(test_rf_pred)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   26.08  112.40  167.94  223.50  270.47 2354.76
```

```r
# nmax = 50 keeps the matrix processing light
test_res_kriged <- krige(
  formula   = rf_residuals ~ 1,
  locations = spatial_train,
  newdata   = spatial_test,
  model     = fitted_vgm,
  nmax      = 50
)
```

```
## [using ordinary kriging]
```

```r
# Extracting the interpolated residuals
kriged_residual_values <- test_res_kriged$var1.pred

# set spatial correction to 0 for unkriged points
kriged_residual_values[is.na(kriged_residual_values)] <- 0

# Combined predictions
spatial_test$final_kriged_pred <- test_rf_pred + kriged_residual_values



# Checks for data gaps

cat("Number of skipped test points remaining:", sum(is.na(spatial_test$final_kriged_pred)), "\n")
```

```
## Number of skipped test points remaining: 0
```

```r
any(is.na(test_res_kriged$var1.pred))
```

```
## [1] FALSE
```

```r
head(test_res_kriged)
```

```
## Simple feature collection with 6 features and 2 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: 144876.2 ymin: -461396.6 xmax: 177485.4 ymax: -431666.2
## Projected CRS: NAD83 / California Albers
##     var1.pred var1.var                   geometry
## 1 -16.1881720 1250.671 POINT (144876.2 -446355.2)
## 2  59.3573140 1216.189 POINT (148610.9 -435823.6)
## 3 -16.9856253 1352.903 POINT (177485.4 -461396.6)
## 4 -10.5757593 1201.671 POINT (160445.7 -432090.1)
## 5  -9.0165823 1229.041 POINT (151584.7 -446494.2)
## 6   0.9773975 1182.295 POINT (154968.5 -431666.2)
```

Final RF and Kriging pred

```r
kriged_values <- as.numeric(test_res_kriged$var1.pred)

# If any point was out of range and skipped, treat correction as 0
kriged_values[is.na(kriged_values)] <- 0

# Combining  RF with the spatial correction
final_test_predictions <- test_rf_pred + kriged_values


test_data1$final_kriged_pred <- final_test_predictions
spatial_test$final_kriged_pred <- final_test_predictions

test.res <- final_test_predictions- test_data1$price


# 2. Model evaluations

### RF training Predictions
eval_RF <- data.frame(
  obs  = test_data1$price,
  pred = test_data1$rf_pred
)

cat("--- Random Forest Evaluation Top Rows ---\n")
```

```
## --- Random Forest Evaluation Top Rows ---
```

```r
print(head(eval_RF))
```

```
##   obs     pred
## 1  55  78.4490
## 2 310 415.5847
## 3  65 121.9892
```

```
## 4 150 176.9857
## 5 124 172.8880
## 6 296 281.9182
```

```
### Combined RF+Kriging Predictions
eval_test <- data.frame(
  obs  = test_data1$price,
  pred = test_data1$final_kriged_pred
)

cat("\n--- Combined RF + Kriging Evaluation Top Rows ---\n")
```

```
##
## --- Combined RF + Kriging Evaluation Top Rows ---
```

```
print(head(eval_test))
```

```
##   obs      pred
## 1  55  62.26083
## 2 310 474.94198
## 3  65 105.00354
## 4 150 166.40991
## 5 124 163.87142
## 6 296 282.89556
```

```
# 1.coordinates matrix from the spatial dataset
coords_test <- st_coordinates(spatial_test)


spatial_test$Easting  <- coords_test[, 1]
spatial_test$Northing <- coords_test[, 2]

# 3.Pricing Heatmap
pdf("Pricing_Heatmap.pdf", width = 6, height = 5 )
ggplot(data = spatial_test) +
  stat_summary_hex(
    aes(x = Easting, y = Northing, z = final_kriged_pred),
    fun = function(x) mean(x, na.rm = TRUE),
    bins = 40
  ) +
  scale_fill_viridis_c(option = "plasma", name = "Avg Price ($)") +
  theme_minimal() +
  labs(
    title = "Spatial Heatmap: Predicted Airbnb Prices (RF+K)",
    subtitle = "Aggregated Regional Price Trends (California)",
    x = "Easting (metres)",
    y = "Northing (metres)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right"
  )
```

```
dev.off()
```

```
## pdf
##   2
```

```
pdf("RFK_error.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = test.res), size = 1.2, alpha = 0.7) +
  # Diverging scale: Red means underpriced, Blue means overpriced, Green is good.
  scale_color_gradient2(
    low = "blue", mid = "green", high = "red",
    midpoint = 0, name = "Prediction Error ($)"
  ) +
  theme_minimal() +
  labs(
    title = "Spatial Error Analysis: Where is the (RF+K) Model Wrong?",
    subtitle = "Red = Predicted Too Low | Blue = Predicted Too High",
    x = "Easting (metres)", y = "Northing (metres)"
  ) +
  theme(plot.title = element_text(face = "bold", size = 14))
dev.off()
```

```
## pdf
##   2
```

```
# Thematic Point Map
pdf("Thematic_Point_Map_RF_pred.pdf", width = 6, height = 5 )
ggplot(data = spatial_train) +
  geom_sf(aes(color = y_pred), size = 2, alpha = 0.8) +
  scale_color_viridis_c(option = "turbo", name = "RF Prediction") +
  labs(
    title = "Spatial Random Forest Predictions",
    subtitle = "Mapped directly from coordinate points"
  ) +
  theme_minimal()

dev.off()
```

```
## pdf
##   2
```

More evaluations

```
evaluate <- function(pred, obs, model_title = "Actual vs Predicted") {

  # FIXED: Explicitly use base::mean to bypass the environment variable conflict (Aid with previous iss
  mse <- base::mean((pred - obs)^2)
  rmse <- sqrt(mse)
  mae <- base::mean(abs(pred - obs))
```

```
  r_squared <- 1 - (sum((obs - pred)^2) / sum((obs - base::mean(obs))^2))

  plot(
    obs, pred,
    main = model_title,
    xlab = "Actual Price ($)",
    ylab = "Predicted Price ($)",
    col  = rgb(0, 0, 1, alpha = 0.2),
    pch  = 19,
    cex  = 0.6
  )


  abline(0, 1, col = "red", lwd = 2)


  cat("---", model_title, "Metrics ---\n")
  cat("MSE:       ", round(mse, 2), "\n")
  cat("RMSE:      ", round(rmse, 2), "\n")
  cat("MAE:       ", round(mae, 2), "\n")
  cat("R-squared: ", round(r_squared, 4), "\n\n")
}
```

```
# 1. Evaluate the baseline Random Forest
pdf("prediction_map1.pdf",width = 6, height = 5 )
evaluate(test_data1$rf_pred, test_data1$price, "Baseline Random Forest")
```

```
## --- Baseline Random Forest Metrics ---
## MSE:        17707.65
## RMSE:       133.07
## MAE:        64.75
## R-squared:  0.6904
```

```
dev.off()
```

```
## pdf
##   2
```

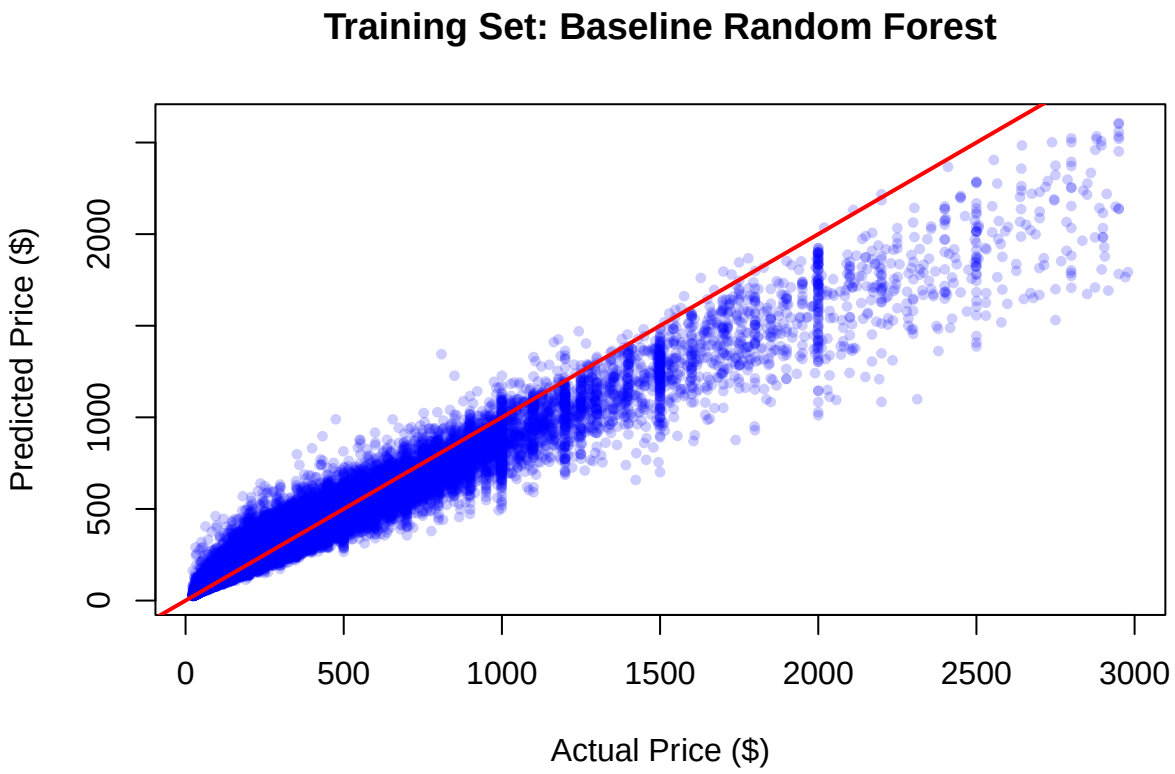
```
# 2. Evaluate the Combined RF + Kriging model
pdf("prediction_map2.pdf",width = 6, height = 5 )
evaluate(test_data1$final_kriged_pred, test_data1$price, "Combined RF + Kriging")
```

```
## --- Combined RF + Kriging Metrics ---
## MSE:        14341.65
## RMSE:       119.76
## MAE:        57.83
## R-squared:  0.7492
```

```
dev.off()
```

```
## pdf
##   2
```

```r
# predictions on the training data from the final rf2 model
train_rf_pred <- predict(rf2, data = train_final)$predictions


train_data1$rf_pred <- train_rf_pred

evaluate(train_data1$rf_pred, train_data1$price, "Training Set: Baseline Random Forest")
```




```
## --- Training Set: Baseline Random Forest Metrics ---
## MSE:         3876.98
## RMSE:        62.27
## MAE:         28.75
## R-squared:   0.9357
```

The pure random forest model

Random forest model will be fitted thus, first the non-numeric variables are removed longitude and latitude included this time.

```r
train_final1<- train_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
          -host_city, -logprice)
```

```r
test_final1 <- test_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
          -host_city, -logprice)
```

Removal of past predictions.

```r
tracking_cols <- c("y_pred", "rf_residuals", "rf_pred", "final_kriged_pred")

# clean training and testing subsets that keep coordinates but drop tracking columns
pure_rf_train <- train_final1[, setdiff(names(train_final1), tracking_cols)]
pure_rf_test  <- test_final1[, setdiff(names(test_final1), tracking_cols)]

head(pure_rf_train)
```

```
##        host_acceptance_rate host_is_superhost host_listings_count
## 14535               -0.0100                 0                   2
## 180050               0.0033                 0                   3
## 69962                0.0055                 1                   1
## 192041               0.0100                 1                   7
## 228008               0.0100                 0                  13
## 186526               0.0100                 0                   3
##        host_total_listings_count host_identity_verified latitude longitude
## 14535                          3                      1 33.83633 -118.3880
## 180050                         3                      0 32.84812 -117.2035
## 69962                          2                      1 37.63295 -122.4488
## 192041                         7                      0 32.84355 -117.2678
## 228008                        27                      1 37.66486 -122.4021
## 186526                         3                      0 32.61261 -117.0585
##        accommodates beds price minimum_nights maximum_nights
## 14535             1    1   110             90            360
## 180050            1    1   125             31           1125
## 69962             4    1   135              4           1125
## 192041            6    3   604              4             60
## 228008            2    1    55              1             28
## 186526           16    8   314             30            365
##        minimum_minimum_nights maximum_minimum_nights minimum_maximum_nights
## 14535                      90                     90                   360
## 180050                     31                     31                  1125
## 69962                       4                      4                  1125
## 192041                      4                      5                  1125
## 228008                      1                      1                    28
## 186526                     30                     30                   365
##        maximum_maximum_nights minimum_nights_avg_ntm maximum_nights_avg_ntm
## 14535                     360                     90                   360
## 180050                   1125                     31                  1125
## 69962                    1125                      4                  1125
## 192041                   1125                      4                  1125
## 228008                     28                      1                    28
## 186526                    365                     30                   365
##        has_availability availability_30 availability_60 availability_90
## 14535                 0               0               0               0
```

```
## 180050               1             27             57             87
## 69962                1              0             19             49
## 192041               1             14             39             53
## 228008               1             25             55             85
## 186526               1              9             21             51
##        availability_365 number_of_reviews number_of_reviews_ltm
## 14535                 0                2                     0
## 180050              362                3                     0
## 69962               324               70                     2
## 192041              277              157                    32
## 228008              175              205                    58
## 186526              105                6                     6
##        number_of_reviews_l30d review_scores_rating review_scores_accuracy
## 14535                       0                5.00                   5.00
## 180050                      0                4.67                   5.00
## 69962                       0                4.86                   4.87
## 192041                      0                4.92                   4.91
## 228008                      1                4.52                   4.69
## 186526                      1                4.33                   4.00
##        review_scores_cleanliness review_scores_checkin
## 14535                       5.00                  5.00
## 180050                      4.67                  5.00
## 69962                       4.84                  4.96
## 192041                      4.89                  4.97
## 228008                      4.39                  4.86
## 186526                      3.83                  5.00
##        review_scores_communication review_scores_location review_scores_value
## 14535                         5.00                   5.00                5.00
## 180050                        5.00                   5.00                5.00
## 69962                         4.96                   4.86                4.89
## 192041                        4.94                   4.94                4.84
## 228008                        4.76                   4.54                4.50
## 186526                        4.83                   4.17                4.50
##        instant_bookable calculated_host_listings_count
## 14535                 0                              2
## 180050                0                              3
## 69962                 0                              1
## 192041                1                              4
## 228008                0                             12
## 186526                0                              2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
## 180050                                           0
## 69962                                            1
## 192041                                           4
## 228008                                           0
## 186526                                           2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
```

```
##        first_review_num last_review_num host_since_num
## 14535             18217           18231          15961
## 180050            18586           19137          17870
## 69962             16906           19602          16340
## 192041            17869           19585          15537
## 228008            17460           19590          17053
## 186526            19322           19517          18054
##        calendar_last_scraped_num Date_num is_new_listing room_typeHotel room
## 14535                      19695    19694              0                    0
## 180050                     19533    19532              0                    0
## 69962                      19713    19789              0                    0
## 192041                     19618    19618              0                    0
## 228008                     19618    19618              0                    0
## 186526                     19532    19532              0                    0
##        room_typePrivate room room_typeShared room Cityoakland Citypacific-grove
## 14535                      0                    0           0                  0
## 180050                     1                    0           0                  0
## 69962                      0                    0           0                  0
## 192041                     0                    0           0                  0
## 228008                     1                    0           0                  0
## 186526                     0                    0           0                  0
##        Citysan-diego Citysan-francisco Citysan-mateo-county
## 14535              0                 0                    0
## 180050             1                 0                    0
## 69962              0                 0                    1
## 192041             1                 0                    0
## 228008             0                 0                    1
## 186526             1                 0                    0
##        Citysanta-clara-county Citysanta-cruz-county has_smoke_alarm has_toaster
## 14535                       0                    0               1           0
## 180050                      0                    0               1           1
## 69962                       0                    0               1           1
## 192041                      0                    0               1           0
## 228008                      0                    0               1           0
## 186526                      0                    0               1           1
##        has_mini_fridge has_hot_water has_dishwasher has_Air_conditioning
## 14535                0             1              1                    0
## 180050               0             1              0                    0
## 69962                1             1              0                    0
## 192041               0             1              1                    0
## 228008               0             0              1                    0
## 186526               0             1              1                    0
##        has_Kitchen has_wifi has__Refrigerator has_Hot_water has_Microwave_
## 14535            0        1                 0            0              0
## 180050           0        1                 0            0              0
## 69962            0        1                 0            0              0
## 192041           0        1                 0            0              0
## 228008           0        1                 0            0              0
## 186526           0        1                 0            0              0
##        host_response_time_num bathrooms_numeric
## 14535                       0              1.0
## 180050                      1              1.0
## 69962                       3              1.0
## 192041                      4              3.5
```

```
## 228008                     4              1.5
## 186526                     4              2.0
```

```
keep_features <- setdiff(names(pure_rf_train), useless_features_auto)



pure_rf_train  <- pure_rf_train[, keep_features]
pure_rf_test   <- pure_rf_test[, keep_features]




## Column check
cat("\nRemaining column count in training set:", ncol(pure_rf_train ), "\n")
```

```
##
## Remaining column count in training set: 40
```

```
head(pure_rf_train )
```

```
##        host_acceptance_rate host_listings_count host_total_listings_count
## 14535               -0.0100                   2                         3
## 180050               0.0033                   3                         3
## 69962                0.0055                   1                         2
## 192041               0.0100                   7                         7
## 228008               0.0100                  13                        27
## 186526               0.0100                   3                         3
##        latitude longitude accommodates beds price minimum_nights maximum_nights
## 14535  33.83633 -118.3880            1    1   110             90            360
## 180050 32.84812 -117.2035            1    1   125             31           1125
## 69962  37.63295 -122.4488            4    1   135              4           1125
## 192041 32.84355 -117.2678            6    3   604              4             60
## 228008 37.66486 -122.4021            2    1    55              1             28
## 186526 32.61261 -117.0585           16    8   314             30            365
##        minimum_minimum_nights maximum_minimum_nights minimum_maximum_nights
## 14535                      90                     90                   360
## 180050                     31                     31                  1125
## 69962                       4                      4                  1125
## 192041                      4                      5                  1125
## 228008                      1                      1                    28
## 186526                     30                     30                   365
##        maximum_maximum_nights minimum_nights_avg_ntm maximum_nights_avg_ntm
## 14535                     360                    90                   360
## 180050                   1125                    31                  1125
## 69962                    1125                     4                  1125
## 192041                   1125                     4                  1125
## 228008                     28                     1                    28
## 186526                    365                    30                   365
##        availability_30 availability_60 availability_90 availability_365
## 14535                0               0              0                0
## 180050              27              57             87              362
## 69962                0              19             49              324
```

```
## 192041              14              39              53              277
## 228008              25              55              85              175
## 186526               9              21              51              105
##        number_of_reviews number_of_reviews_ltm number_of_reviews_l30d
## 14535                  2                     0                     0
## 180050                 3                     0                     0
## 69962                 70                     2                     0
## 192041               157                    32                     0
## 228008               205                    58                     1
## 186526                 6                     6                     1
##        review_scores_rating review_scores_accuracy review_scores_cleanliness
## 14535                  5.00                   5.00                      5.00
## 180050                 4.67                   5.00                      4.67
## 69962                  4.86                   4.87                      4.84
## 192041                 4.92                   4.91                      4.89
## 228008                 4.52                   4.69                      4.39
## 186526                 4.33                   4.00                      3.83
##        review_scores_checkin review_scores_communication review_scores_location
## 14535                   5.00                        5.00                   5.00
## 180050                  5.00                        5.00                   5.00
## 69962                   4.96                        4.96                   4.86
## 192041                  4.97                        4.94                   4.94
## 228008                  4.86                        4.76                   4.54
## 186526                  5.00                        4.83                   4.17
##        review_scores_value calculated_host_listings_count
## 14535                 5.00                              2
## 180050                5.00                              3
## 69962                 4.89                              1
## 192041                4.84                              4
## 228008                4.50                             12
## 186526                4.50                              2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
## 180050                                           0
## 69962                                            1
## 192041                                           4
## 228008                                           0
## 186526                                           2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
##        first_review_num last_review_num host_since_num room_typePrivate room
## 14535             18217           18231          15961                     0
## 180050            18586           19137          17870                     1
## 69962             16906           19602          16340                     0
## 192041            17869           19585          15537                     0
## 228008            17460           19590          17053                     1
## 186526            19322           19517          18054                     0
##        Citysan-diego host_response_time_num
## 14535              0                     0
```

```
## 180050             1                    1
## 69962              0                    3
## 192041             1                    4
## 228008             0                    4
## 186526             1                    4
```

```
#head(pure_rf_train)
```

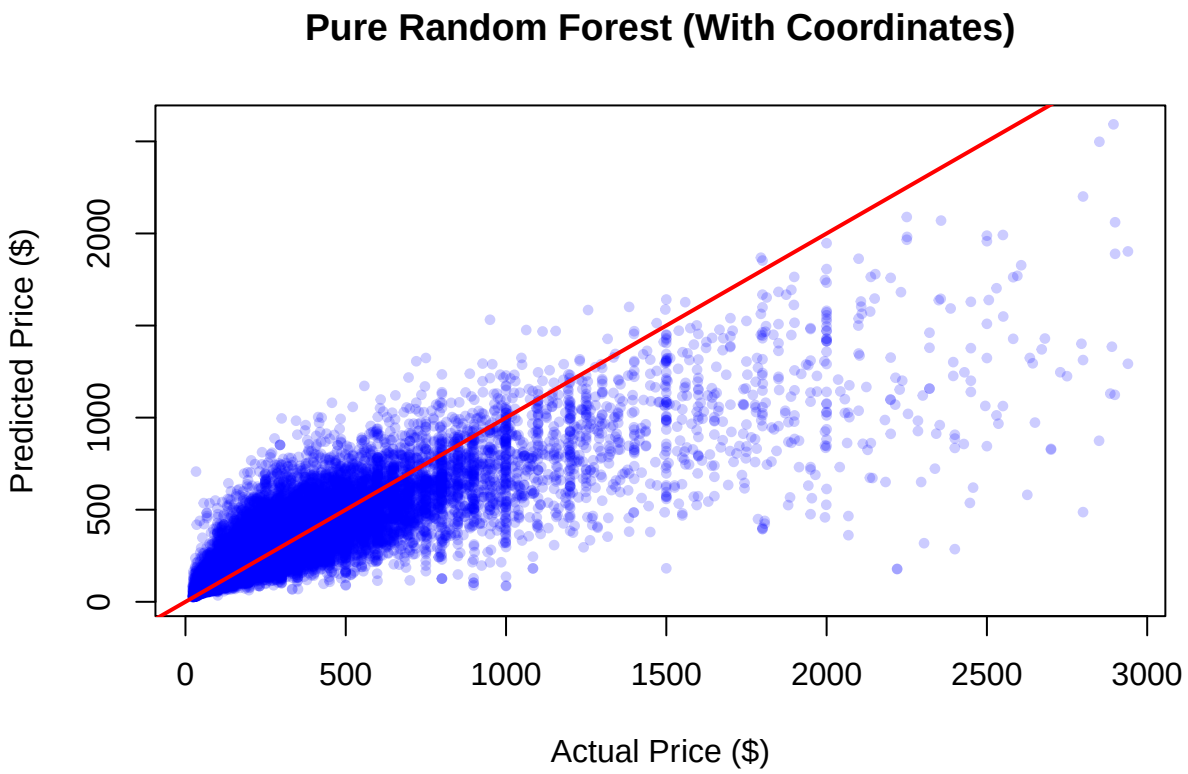
```
set.seed(1852)

pure_rf_model <- ranger(
  dependent.variable.name = "price",
  data                    = pure_rf_train,
  num.trees               = 100,
  importance              = "permutation",
  mtry                    = 8,
  min.node.size           = 5,
  save.memory             = TRUE
)
```

```
## Growing trees.. Progress: 29%. Estimated remaining time: 1 minute, 20 seconds.
## Growing trees.. Progress: 55%. Estimated remaining time: 53 seconds.
## Growing trees.. Progress: 77%. Estimated remaining time: 28 seconds.
## Computing permutation importance.. Progress: 39%. Estimated remaining time: 50 seconds.
## Computing permutation importance.. Progress: 75%. Estimated remaining time: 20 seconds.
```

```
print(pure_rf_model)
```

```
## Ranger result
##
## Call:
##  ranger(dependent.variable.name = "price", data = pure_rf_train,      num.trees = 100, importance =
##
## Type:                             Regression
## Number of trees:                  100
## Sample size:                      129120
## Number of independent variables:  39
## Mtry:                             8
## Target node size:                 5
## Variable importance mode:         permutation
## Splitrule:                        variance
## OOB prediction error (MSE):       15963.84
## R squared (OOB):                  0.735056
```

```
pure_rf_preds <- predict(pure_rf_model, data = pure_rf_test)$predictions


evaluate(pure_rf_preds, pure_rf_test$price, "Pure Random Forest (With Coordinates)")
```




```
## --- Pure Random Forest (With Coordinates) Metrics ---
## MSE:         14809.94
## RMSE:        121.7
## MAE:         58.44
## R-squared:   0.7411
```

```
pure_rf_train_preds <- predict(pure_rf_model, data = pure_rf_train[setdiff(names(pure_rf_train), "price"

pure_rf_test_pred <- predict(pure_rf_model, data= pure_rf_test[setdiff(names(pure_rf_test), "price") ])$

rf_error <- pure_rf_test_pred - pure_rf_test$price

evaluate(pure_rf_train_preds, pure_rf_train$price, "Training Set: Pure Random Forest")
```

```
## --- Training Set: Pure Random Forest Metrics ---
## MSE:        3162.34
## RMSE:       56.23
## MAE:        25.81
## R-squared:  0.9475
```

```
pdf("pure_rf_pred_map.pdf", width = 6, height = 5)
evaluate(pure_rf_test_pred, pure_rf_test$price, "Test set: Pure Random Forest")
```

```
## --- Test set: Pure Random Forest Metrics ---
## MSE:        14809.94
## RMSE:       121.7
## MAE:        58.44
## R-squared:  0.7411
```

```
dev.off()
```

```
## pdf
##   2
```

```
pdf("Pure_RF_heat_map", width=6, height=5)
#coords_test <- st_coordinates(spatial_test)

# coordinates added back as simple columns
```

```r
#spatial_test$Easting  <- coords_test[, 1]
#spatial_test$Northing <- coords_test[, 2]

ggplot(data = spatial_test) +
  stat_summary_hex(
    aes(x = Easting, y = Northing, z = pure_rf_test_pred),
    fun = function(x) mean(x, na.rm = TRUE),
    bins = 40
  ) +
  scale_fill_viridis_c(option = "plasma", name = "Avg Price ($)") +
  theme_minimal() +
  labs(
    title = " Spatial Heatmap: Predicted Pure RF Airbnb Prices",
    subtitle = "Real-World Regional Price Distribution (California)",
    x = "Easting (metres)",
    y = "Northing (metres)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right"
  )

dev.off()
```

```
## pdf
##   2
```

```r
pdf("Actual_price_map.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = price), size = 2, alpha = 0.8) +
  scale_color_viridis_c(option = "turbo", name = "Actual Price ($)") +
  labs(
    title = "Spatial Actual Airbnb Prices",
    subtitle = "Mapped directly from coordinate points"
  ) +
  theme_minimal()
dev.off()
```

```
## pdf
##   2
```

```r
pdf("RF_Error.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = rf_error), size = 1.2, alpha = 0.7) +
  # Diverging scale: Red means underpriced, Blue means overpriced, Green is good.
  scale_color_gradient2(
    low = "blue", mid = "green", high = "red",
    midpoint = 0, name = "Prediction Error ($)"
  ) +
  theme_minimal() +
  labs(
    title = "Spatial Error Analysis: Where is the (RF) Model Wrong?",
```

```
    subtitle = "Red = Predicted Too Low | Blue = Predicted Too High",
    x = "Easting (metres)", y = "Northing (metres)"
  ) +
  theme(plot.title = element_text(face = "bold", size = 14))
dev.off()
```

```
## pdf
##   2
```

Neural Network and Hybrid framework

```
library(neuralnet)
```

```
##
## Attaching package: 'neuralnet'
```

```
## The following object is masked from 'package:dplyr':
##
##     compute
```

```
library(nnet)
library(caret)
library(janitor)
```

```
## Warning: package 'janitor' was built under R version 4.5.3
```

```
##
## Attaching package: 'janitor'
```

```
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
library(sf)
library(gstat)
library(ggplot2)
library(dplyr)
library(scales)
```

```
##
## Attaching package: 'scales'
```

```
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
head(train_data1)
```

```
##                 id host_acceptance_rate host_is_superhost host_listings_count
## 14535  3.908285e+07              -0.0100                0                   2
## 180050 3.230752e+07               0.0033                0                   3
## 69962  8.666864e+06               0.0055                1                   1
## 192041 2.961060e+07               0.0100                1                   7
## 228008 2.132914e+07               0.0100                0                  13
## 186526 7.607613e+17               0.0100                0                   3
##        host_total_listings_count host_identity_verified neighbourhood_cleansed
## 14535                          3                      1          Redondo Beach
## 180050                         3                      0        University City
## 69962                          2                      1              San Bruno
## 192041                         7                      0               La Jolla
## 228008                        27                      1    South San Francisco
## 186526                         3                      0              Southwest
##        latitude longitude        property_type accommodates beds price
## 14535  33.83633 -118.3880   Entire rental unit            1    1   110
## 180050 32.84812 -117.2035 Private room in home            1    1   125
## 69962  37.63295 -122.4488   Entire guest suite            4    1   135
## 192041 32.84355 -117.2678          Entire home            6    3   604
## 228008 37.66486 -122.4021 Private room in home            2    1    55
## 186526 32.61261 -117.0585          Entire home           16    8   314
##        minimum_nights maximum_nights minimum_minimum_nights
## 14535              90            360                    90
## 180050             31           1125                    31
## 69962               4           1125                     4
## 192041              4             60                     4
## 228008              1             28                     1
## 186526             30            365                    30
##        maximum_minimum_nights minimum_maximum_nights maximum_maximum_nights
## 14535                      90                    360                   360
## 180050                     31                   1125                  1125
## 69962                       4                   1125                  1125
## 192041                      5                   1125                  1125
## 228008                      1                     28                    28
## 186526                     30                    365                   365
##        minimum_nights_avg_ntm maximum_nights_avg_ntm has_availability
## 14535                      90                    360                0
## 180050                     31                   1125                1
## 69962                       4                   1125                1
## 192041                      4                   1125                1
## 228008                      1                     28                1
## 186526                     30                    365                1
##        availability_30 availability_60 availability_90 availability_365
## 14535                0               0               0                0
## 180050              27              57              87              362
## 69962                0              19              49              324
## 192041              14              39              53              277
## 228008              25              55              85              175
```

```
## 186526               9              21              51              105
##        number_of_reviews number_of_reviews_ltm number_of_reviews_l30d
## 14535                  2                     0                      0
## 180050                 3                     0                      0
## 69962                 70                     2                      0
## 192041               157                    32                      0
## 228008               205                    58                      1
## 186526                 6                     6                      1
##        review_scores_rating review_scores_accuracy review_scores_cleanliness
## 14535                  5.00                   5.00                      5.00
## 180050                 4.67                   5.00                      4.67
## 69962                  4.86                   4.87                      4.84
## 192041                 4.92                   4.91                      4.89
## 228008                 4.52                   4.69                      4.39
## 186526                 4.33                   4.00                      3.83
##        review_scores_checkin review_scores_communication review_scores_location
## 14535                   5.00                        5.00                   5.00
## 180050                  5.00                        5.00                   5.00
## 69962                   4.96                        4.96                   4.86
## 192041                  4.97                        4.94                   4.94
## 228008                  4.86                        4.76                   4.54
## 186526                  5.00                        4.83                   4.17
##        review_scores_value instant_bookable calculated_host_listings_count
## 14535                 5.00                0                              2
## 180050                5.00                0                              3
## 69962                 4.89                0                              1
## 192041                4.84                1                              4
## 228008                4.50                0                             12
## 186526                4.50                0                              2
##        calculated_host_listings_count_entire_homes
## 14535                                            2
## 180050                                           0
## 69962                                            1
## 192041                                           4
## 228008                                           0
## 186526                                           2
##        calculated_host_listings_count_private_rooms reviews_per_month
## 14535                                             0              0.04
## 180050                                            3              0.09
## 69962                                             0              0.75
## 192041                                            0              2.69
## 228008                                           12              2.85
## 186526                                            0              0.85
##           host_city first_review_num last_review_num host_since_num
## 14535  Redondo Beach            18217           18231          15961
## 180050     San Diego            18586           19137          17870
## 69962      San Bruno            16906           19602          16340
## 192041     San Diego            17869           19585          15537
## 228008     Daly City            17460           19590          17053
## 186526                          19322           19517          18054
##        calendar_last_scraped_num Date_num is_new_listing room_typeHotel room
## 14535                      19695    19694              0                   0
## 180050                     19533    19532              0                   0
## 69962                      19713    19789              0                   0
```

```
## 192041                    19618    19618             0                   0
## 228008                    19618    19618             0                   0
## 186526                    19532    19532             0                   0
##        room_typePrivate room room_typeShared room Cityoakland Citypacific-grove
## 14535                     0                   0          0                  0
## 180050                    1                   0          0                  0
## 69962                     0                   0          0                  0
## 192041                    0                   0          0                  0
## 228008                    1                   0          0                  0
## 186526                    0                   0          0                  0
##        Citysan-diego Citysan-francisco Citysan-mateo-county
## 14535              0                 0                    0
## 180050             1                 0                    0
## 69962              0                 0                    1
## 192041             1                 0                    0
## 228008             0                 0                    1
## 186526             1                 0                    0
##        Citysanta-clara-county Citysanta-cruz-county has_smoke_alarm has_toaster
## 14535                       0                    0              1           0
## 180050                      0                    0              1           1
## 69962                       0                    0              1           1
## 192041                      0                    0              1           0
## 228008                      0                    0              1           0
## 186526                      0                    0              1           1
##        has_mini_fridge has_hot_water has_dishwasher has_Air_conditioning
## 14535                0             1              1                    0
## 180050               0             1              0                    0
## 69962                1             1              0                    0
## 192041               0             1              1                    0
## 228008               0             0              1                    0
## 186526               0             1              1                    0
##        has_Kitchen has_wifi has__Refrigerator has_Hot_water has_Microwave_
## 14535            0        1                 0             0              0
## 180050           0        1                 0             0              0
## 69962            0        1                 0             0              0
## 192041           0        1                 0             0              0
## 228008           0        1                 0             0              0
## 186526           0        1                 0             0              0
##        host_response_time_num bathrooms_numeric logprice    y_pred rf_residuals
## 14535                       0              1.0 4.700480 109.92400     0.076000
## 180050                      1              1.0 4.828314 110.25117    14.748833
## 69962                       3              1.0 4.905275 139.17350    -4.173500
## 192041                      4              3.5 6.403574 491.05767   112.942333
## 228008                      4              1.5 4.007333  56.50242    -1.502417
## 186526                      4              2.0 5.749393 394.03133   -80.031333
##          rf_pred
## 14535  109.92400
## 180050 110.25117
## 69962  139.17350
## 192041 491.05767
## 228008  56.50242
## 186526 394.03133
```

```
## Unneeded variables
train_nn <- train_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type, -host_city, -logprice)
test_nn  <- test_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type, -host_city, -logprice)
```

```
tracking_cols <- c("y_pred", "rf_residuals", "rf_pred", "final_kriged_pred",
                   "bathrooms_numeric", "has_Microwave_", "has_Hot_water",
                   "has__Refrigerator", "has_Kitchen", "has_Air_conditioning",
                   "has_mini_fridge", "has_toaster", "is_new_listing", "has_dishwasher")

ts_train1 <- train_nn[, setdiff(names(train_nn), tracking_cols)]
ts_test1  <- test_nn[, setdiff(names(test_nn), tracking_cols)]

# Range transformation normalisation preprocessing
preprocess_params <- preProcess(ts_train1, method = c("range"))
scaled_train <- predict(preprocess_params, ts_train1)
scaled_test <- predict(preprocess_params, ts_test1)

## Cleaning the column names
scaled_train <- clean_names(scaled_train)
scaled_test  <- clean_names(scaled_test)
```

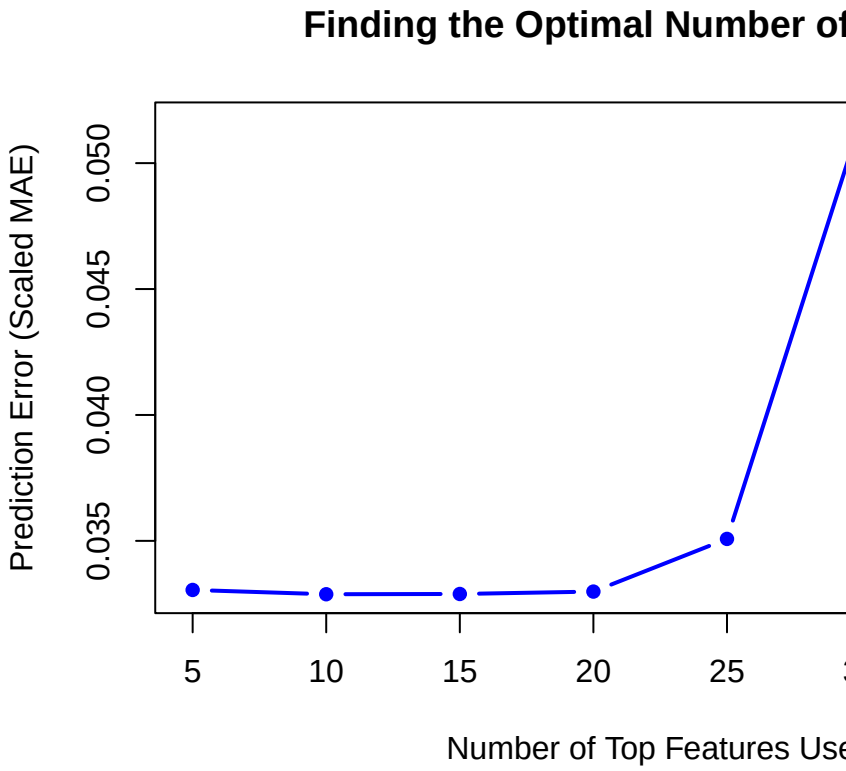
Feature selection and model selection

Lasso Regression (importance filter)

```


numeric_features <- scaled_train[, sapply(scaled_train, is.numeric)]
numeric_features$price <- NULL

corr_matrix       <- cor(numeric_features, use = "pairwise.complete.obs")
highly_correlated <- findCorrelation(corr_matrix, cutoff = 0.85, names = TRUE)
train_filtered    <- scaled_train[, !names(scaled_train) %in% highly_correlated]

# Run Cross-Validated Lasso importance analysis
x_matrix <- model.matrix(price ~ ., data = train_filtered)[, -1]
y_vector <- train_filtered$price
cv_lasso <- cv.glmnet(x_matrix, y_vector, alpha = 1)

coef_matrix <- as.matrix(coef(cv_lasso, s = "lambda.min"))
coef_df     <- data.frame(Feature = rownames(coef_matrix), Importance = abs(coef_matrix[, 1]))
coef_df     <- coef_df[coef_df$Feature != "(Intercept)", ]
```

```


set.seed(1852)
train_sample <- scaled_train[sample(1:nrow(scaled_train), 5000), ]
```

```r
set.seed(1852)
sorted_features   <- coef_df[order(-coef_df$Importance), "Feature"]
feature_counts    <- c(5, 10, 15, 20, 25, 30, 40)
validation_errors <- numeric(length(feature_counts))

for (i in seq_along(feature_counts)) {
  k                <- feature_counts[i]
  current_features <- sorted_features[1:k]
  temp_formula     <- as.formula(paste("price ~", paste(current_features, collapse = " + ")))
  hidden_neurons   <- max(3, round(k / 2))

  suppressWarnings({
    temp_model <- neuralnet(
      temp_formula, data = train_sample, hidden = c(hidden_neurons),
      linear.output = TRUE, stepmax = 5e5, threshold = 0.1
    )
    test_feats         <- scaled_test[, current_features, drop = FALSE]
    preds              <- as.vector(compute(temp_model, test_feats)$net.result)
    validation_errors[i] <- mean(abs(preds - scaled_test$price))
  })
}

performance_df <- data.frame(Features = feature_counts, MAE = validation_errors)
plot(performance_df$Features, performance_df$MAE, type = "b", col = "blue", pch = 16, lwd = 2,
     xlab = "Number of Top Features Used", ylab = "Prediction Error (Scaled MAE)", main = "Finding the
```



This is to select the variable subset for later
20 variables will be enough

```
# Capture top 20 features for tuning criteria
top_20_features <- coef_df[order(-coef_df$Importance), ][1:20, "Feature"]
print("Top 20 Features for the Neural Network are:")
```

```
## [1] "Top 20 Features for the Neural Network are:"
```

```
print(top_20_features)
```

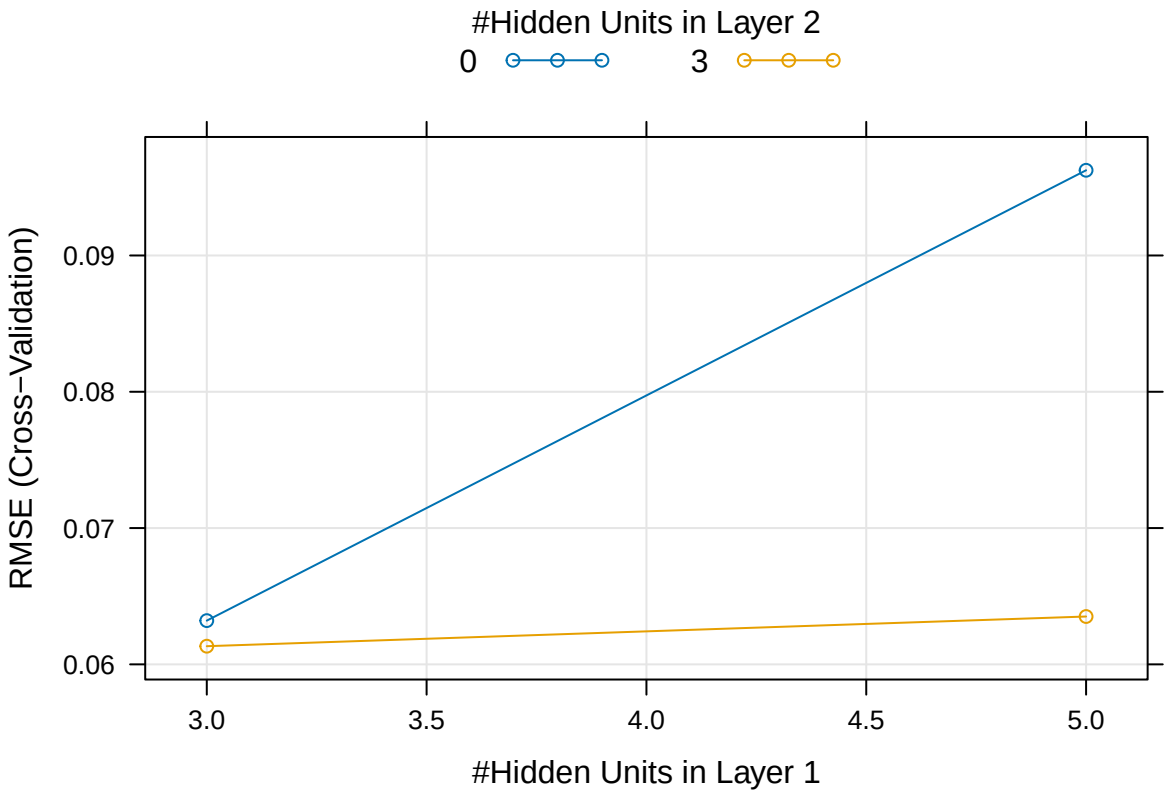
```
##  [1] "reviews_per_month"
##  [2] "accommodates"
##  [3] "minimum_nights"
##  [4] "review_scores_location"
##  [5] "review_scores_value"
##  [6] "review_scores_cleanliness"
##  [7] "citypacific_grove"
##  [8] "maximum_nights_avg_ntm"
##  [9] "review_scores_rating"
## [10] "citysanta_cruz_county"
## [11] "review_scores_communication"
## [12] "review_scores_checkin"
## [13] "calculated_host_listings_count_private_rooms"
## [14] "room_type_shared_room"
## [15] "host_total_listings_count"
## [16] "last_review_num"
```

```
## [17] "availability_30"
## [18] "citysan_francisco"
## [19] "host_since_num"
## [20] "calendar_last_scraped_num"
```

```
set.seed(1852)
train_sample <- scaled_train[sample(1:nrow(scaled_train), 5000), ]


formula_20   <- as.formula(paste("price ~", paste(top_20_features, collapse = " + ")))
fast_control <- trainControl(method = "cv", number = 3)




grid_20 <- expand.grid(layer1 = c(3, 5), layer2 = c(0, 3), layer3 = c(0))

nn_tuned <- train(
  formula_20, data = train_sample, method = "neuralnet",
  trControl = fast_control, tuneGrid = grid_20,
  linear.output = TRUE, stepmax = 1e6, threshold = 0.1
)

print(nn_tuned$bestTune)
```

```
##   layer1 layer2 layer3
## 2      3      3      0
```

```
plot(nn_tuned)
```



3 neuron in layer 1 and 3 in layer 2

```
set.seed(1852)
scaled_train_small <- scaled_train[sample(1:nrow(scaled_train), 5000), ]
feature_names <- top_20_features
nn_formula <- as.formula(paste("price ~", paste(feature_names , collapse = " + ")))

n <- neuralnet(
  formula = nn_formula, data = scaled_train_small, hidden = c(3, 3),
  linear.output = TRUE, threshold = 0.2, stepmax = 10000, lifesign = "minimal"
)
```

```
## hidden: 3, 3    thresh: 0.2    rep: 1/1    steps:     262    error: 9.5683   time: 0.86 secs
```

```
plot(n)



scaled_t  <- scaled_test %>% select(all_of(top_20_features), price)
```

```r
scaled_predictions <- compute(n, scaled_t[, setdiff(names(scaled_t), "price")])$net.result

min_price <- min(ts_train1$price, na.rm = TRUE)
max_price <- max(ts_train1$price, na.rm = TRUE)

actual_predictions <- scaled_predictions*(max_price - min_price) + min_price

actual_test_prices <- ts_test1$price
ts_test1$n_pred    <- actual_predictions

# Calculate Evaluation Metrics
error <- actual_test_prices - actual_predictions


mae <- mean(abs(error))

rmse <- sqrt(mean(error^2))

# R-squared calculation
ss_res <- sum(error^2)
ss_tot <- sum((actual_test_prices - mean(actual_test_prices))^2)
r_squared <- 1 - (ss_res / ss_tot)

# Print performance scorecard
cat("--- NEURAL NETWORK PERFORMANCE RECORD ---\n",
    "Mean Absolute Error (MAE):     $", round(mae, 2), "\n",
    "Root Mean Squared Error (RMSE): $", round(rmse, 2), "\n",
    "R-squared Score (R²):           ", round(r_squared, 4), "\n",
    "----------------------------------------\n")
```

```
## --- NEURAL NETWORK PERFORMANCE RECORD ---
##  Mean Absolute Error (MAE):     $ 97.15
##  Root Mean Squared Error (RMSE): $ 185.71
##  R-squared Score (R²):            0.397
##  ----------------------------------------
```

```r
results_df <- data.frame(Actual = actual_test_prices, Predicted = as.vector(actual_predictions))

pdf("NN_Test_pred.pdf", width=6, height=5)
plot(results_df$Actual, results_df$Predicted, xlab = "Actual Prices ($)", ylab = "Predicted Prices ($)"
     main = "Testset Neural Network Regression : Predicted vs Actual ", col = alpha("blue", 0.5), pch =
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
dev.off
```

```
## function (which = dev.cur())
## {
##     if (which == 1)
##         stop("cannot shut down device 1 (the null device)")
##     .External(C_devoff, as.integer(which))
##     dev.cur()
## }
```

```
## <bytecode: 0x0000026aa575fa18>
## <environment: namespace:grDevices>
```

```
pdf("NN_Error.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = error), size = 1.2, alpha = 0.7) +
  # Diverging scale: Red means underpriced, Blue means overpriced, Green is good.
  scale_color_gradient2(
    low = "blue", mid = "green", high = "red",
    midpoint = 0, name = "Prediction Error ($)"
  ) +
  theme_minimal() +
  labs(
    title = "Spatial Error Analysis: Where is the (NN) Model Wrong?",
    subtitle = "Red = Predicted Too Low | Blue = Predicted Too High",
    x = "Easting (metres)", y = "Northing (metres)"
  ) +
  theme(plot.title = element_text(face = "bold", size = 14))
dev.off()
```

```
## pdf
##   2
```

NN + Kriging

```
scaled_train1 <- scaled_train %>% select(all_of(top_20_features), -latitude, price)
scaled_test1  <- scaled_test %>% select(all_of(top_20_features), -latitude, price)

scaled_train1_small <- scaled_train1[sample(1:nrow(scaled_train1), 5000), ]

set.seed(1852)
feature_names <- setdiff(names(scaled_train1), "price")
nn_formula <- as.formula(paste("price ~", paste(feature_names, collapse = " + ")))

n1 <- neuralnet(
  formula = nn_formula, data = scaled_train1_small, hidden = c(3, 3),
  linear.output = TRUE, threshold = 0.2, stepmax = 10000
)
plot(n1)
```

```
scaled_predictions1 <- compute(n1, scaled_train1[, setdiff(names(scaled_train1), "price")])$net.result


actual_predictions1 <- scaled_predictions1 * (max_price - min_price) + min_price
actual_train_prices <- ts_train1$price

error_train <- actual_train_prices - as.vector(actual_predictions1)
mae_tr  <- mean(abs(error_train))
rmse_tr <- sqrt(mean(error_train^2))
r_squared_tr <- 1 - (sum(error_train^2) / sum((actual_train_prices - mean(actual_train_prices))^2))
```

```r
cat("--- NEURAL NETWORK TRAINING RESIDUAL PERFORMANCE RECORD ---\n",
    "Mean Absolute Error (MAE):     $", round(mae_tr, 2), "\n",
    "Root Mean Squared Error (RMSE): $", round(rmse_tr, 2), "\n",
    "R-squared Score (R²):           ", round(r_squared_tr, 4), "\n")
```

```
## --- NEURAL NETWORK TRAINING RESIDUAL PERFORMANCE RECORD ---
##  Mean Absolute Error (MAE):     $ 97.38
##  Root Mean Squared Error (RMSE): $ 188.3
##  R-squared Score (R²):            0.4115
```

```r
scaled_train$y_pred <- as.vector(actual_predictions1)
scaled_train$n_residuals <- error_train
```

Residual check for normality

```r
pdf("NN_Residual_Normality_Diagnostics.pdf", width = 10, height = 8)

# Set up a 2x2 grid framework for side-by-side visual analysis
par(mfrow = c(1, 2))

nn_res <- scaled_train$n_residuals

# Plot 1: Histogram of Baseline RF Residuals
hist(nn_res, col = "skyblue", main = "Histogram: Baseline RF Residuals",
     xlab = "Residual Error ($)", border = "white")
abline(v = mean(nn_res, na.rm=TRUE), col = "red", lwd = 2, lty = 2)

# Plot 2: Normal Q-Q Plot of Baseline RF Residuals
qqnorm(nn_res, main = "Q-Q Plot: Baseline RF Residuals", col = rgb(0,0,1, alpha=0.15), pch=19)
qqline(nn_res, col = "red", lwd = 2)



# Reset plotting layout parameters
par(mfrow = c(1, 1))
dev.off()
```

```
## pdf
##   2
```

```r
spatial_train_geo <- st_as_sf(scaled_train, coords = c("longitude", "latitude"), crs = 4326)
spatial_test_geo  <- st_as_sf(scaled_test, coords = c("longitude", "latitude"), crs = 4326)

spatial_train_projected <- st_transform(spatial_train_geo, crs = 3310)
spatial_test_projected  <- st_transform(spatial_test_geo, crs = 3310)

spatial_train <- st_jitter(spatial_train_projected, amount = 1.0)
spatial_test  <- st_jitter(spatial_test_projected, amount = 1.0)

any(is.na(spatial_train$n_residuals))
```

```
## [1] FALSE
```

```
any(is.na(spatial_test$price))
```

```
## [1] FALSE
```

```
sub_sample_size   <- min(5000, nrow(spatial_train))
spatial_train_sub <- spatial_train[sample(1:nrow(spatial_train), sub_sample_size), ]

emp_vgm <- variogram(n_residuals ~ 1, data = spatial_train_sub)
init_variance <- var(spatial_train_sub$n_residuals, na.rm = TRUE)

init_vgm <- vgm(
  psill  = init_variance * 0.7,
  model  = "Sph",
  range  = 5000,
  nugget = init_variance * 0.3
)

fitted_vgm <- fit.variogram(emp_vgm, model = init_vgm)
```

```
## Warning in fit.variogram(emp_vgm, model = init_vgm): singular model in
## variogram fit
```

```
print(fitted_vgm)
```

```
##   model    psill range
## 1   Nug 10836.51     0
## 2   Sph 25285.19  5000
```

```
pdf("NN_variogram.pdf", width = 6, height = 5)
plot(emp_vgm, model = fitted_vgm, xlab = "Distance (metres)", ylab = "Semivariance",
     main  = list(label = "Fitted Semivariogram Model of Price NN Residuals", cex = 1.2))

dev.off()
```

```
## pdf
##   2
```

```
scaled_t_predictions <- compute(n1, scaled_test1[, setdiff(names(scaled_test1), "price")])$net.result
actual_predictions   <- scaled_t_predictions * (max_price - min_price) + min_price
actual_test_prices   <- ts_test1$price

scaled_test$n_pred <- as.vector(actual_predictions)


test_res_kriged <- krige(
  formula = n_residuals ~ 1, locations = spatial_train,
  newdata = spatial_test, model = fitted_vgm, nmax = 50
)
```

```
## [using ordinary kriging]
```

```
kriged_residual_values <- test_res_kriged$var1.pred
kriged_residual_values[is.na(kriged_residual_values)] <- 0

spatial_test$final_kriged_pred <- as.vector(actual_predictions) + kriged_residual_values
cat("Number of skipped test points remaining:", sum(is.na(spatial_test$final_kriged_pred)), "\n")
```

```
## Number of skipped test points remaining: 0
```

```
kriged_values <- as.numeric(test_res_kriged$var1.pred)

kriged_values[is.na(kriged_values)] <- 0

final_test_predictions <- as.vector(actual_predictions) + kriged_values

scaled_test$final_kriged_pred <- final_test_predictions

spatial_test$final_kriged_pred <- final_test_predictions




nk_test_res <- final_test_predictions - actual_test_prices



eval_NN <- data.frame(obs = actual_train_prices, pred = scaled_train$y_pred)
print(head(eval_NN))
```

```
##   obs        pred
## 1 110   0.5593639
## 2 125  83.5004899
## 3 135 197.8816757
## 4 604 296.3483751
## 5  55 109.8609262
## 6 314 862.7304465
```

```
eval_test <- data.frame(obs = ts_test1$price, pred = scaled_test$final_kriged_pred)
print(head(eval_test))
```

```
##   obs      pred
## 1  55   3.65169
## 2 310 516.83862
## 3  65  91.94889
## 4 150 114.74419
## 5 124 166.75094
## 6 296 180.84504
```

```
# 1.coordinates matrix from the spatial dataset
coords_test <- st_coordinates(spatial_test)
```

```r
spatial_test$Easting  <- coords_test[, 1]
spatial_test$Northing <- coords_test[, 2]

# 3.Pricing Heatmap
pdf("Pricing_Heatmap.pdf", width = 6, height = 5 )
ggplot(data = spatial_test) +
  stat_summary_hex(
    aes(x = Easting, y = Northing, z = final_kriged_pred),
    fun = function(x) mean(x, na.rm = TRUE),
    bins = 40
  ) +
  scale_fill_viridis_c(option = "plasma", name = "Avg Price ($)") +
  theme_minimal() +
  labs(
    title = "Spatial Heatmap: Predicted Airbnb Prices (RF+K)",
    subtitle = "Aggregated Regional Price Trends (California)",
    x = "Easting (metres)",
    y = "Northing (metres)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right"
  )

dev.off()
```

```
## pdf
##   2
```

```r
#coords_test <- st_coordinates(spatial_test)
#spatial_test$Easting  <- coords_test[, 1]
#spatial_test$Northing <- coords_test[, 2]

# 1. Export the Spatial Pricing Heatmap to PDF
pdf("spatial_nn_heatmap.pdf", width = 7, height = 5)
ggplot(data = spatial_test) +
  stat_summary_hex(aes(x = Easting, y = Northing, z = final_kriged_pred), fun = function(x) mean(x, na.
  scale_fill_viridis_c(option = "plasma", name = "Avg Price ($)") +
  theme_minimal() +
  labs(title = "Spatial Heatmap: NN+OK Predicted Airbnb Prices", subtitle = "Aggregated Regional Price
dev.off()
```

```
## pdf
##   2
```

```r
evaluate_and_save <- function(pred, obs, model_title = "Actual vs Predicted", file_name = "diagnostic_pl
  mse       <- base::mean((pred - obs)^2)
  rmse      <- sqrt(mse)
  mae       <- base::mean(abs(pred - obs))
  r_squared <- 1 - (sum((obs - pred)^2) / sum((obs - base::mean(obs))^2))
```

```
  pdf(file_name, width = 6, height = 6)

  plot(obs, pred, main = model_title, xlab = "Actual Price ($)", ylab = "Predicted Price ($)",
       col = rgb(0, 0, 1, alpha = 0.2), pch = 19, cex = 0.6)
  abline(0, 1, col = "red", lwd = 2)


  dev.off()

  # Print scorecard metrics to the R console
  cat("---", model_title, "Metrics ---\n")
  cat("MSE:        ", round(mse, 2), "\n")
  cat("RMSE:       ", round(rmse, 2), "\n")
  cat("MAE:        ", round(mae, 2), "\n")
  cat("R-squared:  ", round(r_squared, 4), "\n\n")
}

evaluate_and_save(scaled_test$n_pred, ts_test1$price, "Baseline NN", "diagnostic_baseline_nn.pdf")
```

```
## --- Baseline NN Metrics ---
## MSE:        34005.93
## RMSE:       184.41
## MAE:        96.77
## R-squared:  0.4054
```

```
evaluate_and_save(scaled_test$final_kriged_pred, ts_test1$price, "Combined NN + Kriging", "diagnostic_n
```

```
## --- Combined NN + Kriging Metrics ---
## MSE:        23969.58
## RMSE:       154.82
## MAE:        85.64
## R-squared:  0.5809
```

```
pdf("NNK_Error.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = nk_test_res), size = 1.2, alpha = 0.7) +
  # Diverging scale: Red means underpriced, Blue means overpriced, Green is good.
  scale_color_gradient2(
    low = "blue", mid = "green", high = "red",
    midpoint = 0, name = "Prediction Error ($)"
  ) +
  theme_minimal() +
  labs(
    title = "Spatial Error Analysis: Where is the (NN+K) Model Wrong?",
    subtitle = "Red = Predicted Too Low | Blue = Predicted Too High",
    x = "Easting (metres)", y = "Northing (metres)"
  ) +
  theme(plot.title = element_text(face = "bold", size = 14))
dev.off()
```

```
## pdf
##   2
```

Regression-Kriging

```
library(sf)

### longitude and latitude included only in kriging model.

train_rk<-train_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
          -host_city, -logprice)
test_rk <- test_data1 %>%
   select(-id, -neighbourhood_cleansed, -property_type,
         -host_city, -logprice)

## Removal of past predictions.
tracking_cols <- c("y_pred", "rf_residuals", "rf_pred",
                   "final_kriged_pred", "bathrooms_numeric",
                   "has_Microwave_",
                      "has_Hot_water", "has__Refrigerator",
                      "has_Kitchen", "has_Air_conditioning",
                       "has_mini_fridge", "has_toaster",
                      "is_new_listing","has_toaster",
                       "has_dishwasher")

# clean training and testing subsets that keep coordinates but drop tracking columns
rk_train <- train_rk[, setdiff(names(train_rk), tracking_cols)]

rk_test  <- test_rk[, setdiff(names(test_rk), tracking_cols)]

## Fixing some format issues

if("_listings_count" %in% names(rk_train)) {
  names(rk_train)[names(rk_train) == "_listings_count"] <- "listings_count"
}

if("_listings_count" %in% names(rk_test)) {
  names(rk_test)[names(rk_test) == "_listings_count"] <- "listings_count"
}
```

```
library(janitor)

rk_train <- clean_names(rk_train)
rk_test <- clean_names(rk_test)

rk_train1 <- rk_train[, setdiff(names(rk_train), c("latitude", "longitude"))]
rk_test1 <- rk_test[, setdiff(names(rk_test), c("latitude", "longitude"))]
```

```
library(glmnet)

set.seed(52)

sub_sample_size<- min(5000, nrow(rk_train1))
train_sample <- rk_train1[sample(1:nrow(rk_train1), sub_sample_size), ]
```

```r
sum(is.na(train_sample))
```

```
## [1] 0
```

```r
# 1. Prepare inputs (Lasso requires a matrix format)
x_matrix <- as.matrix(train_sample[, setdiff(names(train_sample), "price")])
y_vector <- train_sample$price

# 2. Run cross-validated Lasso
cv_lasso <- cv.glmnet(x_matrix, y_vector, alpha = 1)

# 3. Extract the features with non-zero coefficients
lasso_coefs <- coef(cv_lasso, s = "lambda.min")
selected_features <- rownames(lasso_coefs)[as.matrix(lasso_coefs) != 0]
selected_features <- setdiff(selected_features, "(Intercept)")

print(paste("Lasso selected", length(selected_features), "features."))
```

```
## [1] "Lasso selected 50 features."
```

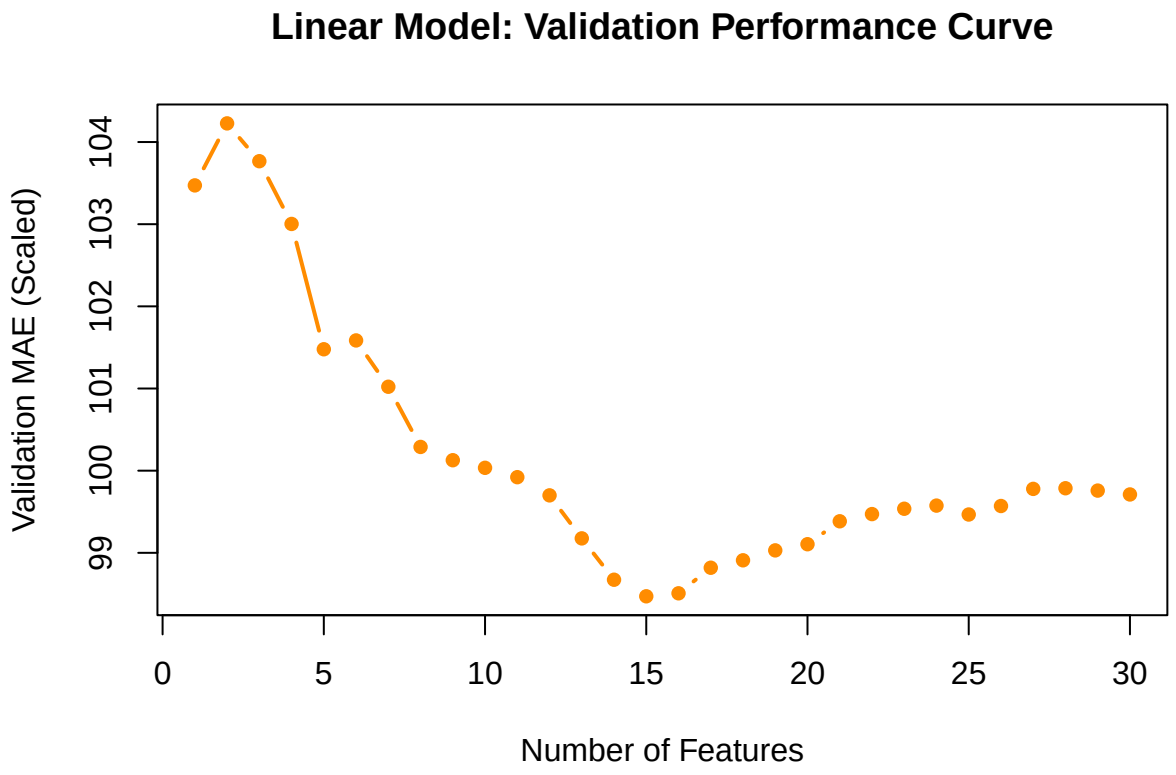
```r
print(selected_features)
```

```
##  [1] "host_acceptance_rate"
##  [2] "host_is_superhost"
##  [3] "host_listings_count"
##  [4] "host_total_listings_count"
##  [5] "host_identity_verified"
##  [6] "accommodates"
##  [7] "beds"
##  [8] "minimum_nights"
##  [9] "maximum_nights"
## [10] "minimum_minimum_nights"
## [11] "maximum_minimum_nights"
## [12] "minimum_maximum_nights"
## [13] "maximum_maximum_nights"
## [14] "minimum_nights_avg_ntm"
## [15] "has_availability"
## [16] "availability_30"
## [17] "availability_60"
## [18] "availability_90"
## [19] "availability_365"
## [20] "number_of_reviews"
## [21] "number_of_reviews_ltm"
## [22] "number_of_reviews_l30d"
## [23] "review_scores_rating"
## [24] "review_scores_accuracy"
## [25] "review_scores_cleanliness"
## [26] "review_scores_checkin"
## [27] "review_scores_communication"
## [28] "review_scores_location"
## [29] "review_scores_value"
```

```
## [30] "instant_bookable"
## [31] "calculated_host_listings_count_entire_homes"
## [32] "calculated_host_listings_count_private_rooms"
## [33] "reviews_per_month"
## [34] "last_review_num"
## [35] "host_since_num"
## [36] "calendar_last_scraped_num"
## [37] "date_num"
## [38] "room_type_hotel_room"
## [39] "room_type_private_room"
## [40] "room_type_shared_room"
## [41] "cityoakland"
## [42] "citypacific_grove"
## [43] "citysan_francisco"
## [44] "citysan_mateo_county"
## [45] "citysanta_clara_county"
## [46] "citysanta_cruz_county"
## [47] "has_smoke_alarm"
## [48] "has_hot_water"
## [49] "has_wifi"
## [50] "host_response_time_num"
```

```r
library(leaps)

set.seed(42)

# 1. Split your train_sample into a 70/30 split
train_idx  <- sample(seq_len(nrow(train_sample)), size = 0.7 * nrow(train_sample))
sub_train  <- train_sample[train_idx, ]
sub_valid  <- train_sample[-train_idx, ]

# 2. Bound your max features dynamically based on what Lasso found
max_features <- min(30, length(selected_features))

# 3. Fit a forward stepwise path on the sub-training block
fit_forward <- regsubsets(
  price ~ .,
  data = sub_train[, c("price", selected_features)],
  nvmax = max_features,
  method = "forward"
)

# 4. Set up an array to capture the validation errors
validation_mae <- numeric(max_features)

# 5. Calculate validation error for each feature count model
for (i in 1:max_features) {
  coef_vals <- coef(fit_forward, id = i)

  # Build a matrix out of the validation data
  valid_matrix <- model.matrix(price ~ ., data = sub_valid[, c("price", selected_features)])
  chosen_vars  <- names(coef_vals)

  # Generate predictions using matrix math
```

```r
  predictions  <- valid_matrix[, chosen_vars] %*% coef_vals

  # Calculate MAE on validation data
  validation_mae[i] <- mean(abs(predictions - sub_valid$price))
}

# 6. Organize into a summary dataframe
performance_df <- data.frame(
  Features = 1:max_features,
  MAE = validation_mae
)


plot(performance_df$Features, performance_df$MAE, type = "b",
     col = "darkorange", pch = 16, lwd = 2,
     xlab = "Number of Features",
     ylab = "Validation MAE (Scaled)",
     main = "Linear Model: Validation Performance Curve")
```



15 features will be enough

```r
# 1.Names of the 15 features from  best forward selection model
best_coef_vals <- coef(fit_forward, id = min(15, max_features))
```

```r
winning_features <- names(best_coef_vals)

# Remove the "(Intercept)" term so we only have the actual variable names
winning_features <- setdiff(winning_features, "(Intercept)")
print(winning_features)
```

```
##  [1] "accommodates"                "beds"
##  [3] "availability_30"             "review_scores_cleanliness"
##  [5] "review_scores_communication" "review_scores_location"
##  [7] "review_scores_value"         "reviews_per_month"
##  [9] "host_since_num"              "calendar_last_scraped_num"
## [11] "room_type_private_room"      "room_type_shared_room"
## [13] "citypacific_grove"           "citysan_francisco"
## [15] "citysanta_cruz_county"
```

```r
rk_train1 <- rk_train1 %>%
  select(all_of(winning_features), price)
rk_test1 <- rk_test1 %>%
  select(all_of(winning_features), price)



# 1. Build the final formula
final_lm_formula <- as.formula(paste("price ~", paste(winning_features, collapse = " + ")))

# 2. Fit the linear model to ALL 100,000+ rows
base_lmod <- lm(final_lm_formula, data = rk_train1)

# 3. View the model coefficients and R-squared score
summary(base_lmod)
```

```
##
## Call:
## lm(formula = final_lm_formula, data = rk_train1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -856.53  -76.65  -21.07   34.75 2725.65
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   3.195e+03  1.381e+02   23.13   <2e-16 ***
## accommodates                  4.126e+01  3.975e-01  103.80   <2e-16 ***
## beds                          1.766e+01  6.176e-01   28.59   <2e-16 ***
## availability_30               1.584e+00  4.809e-02   32.95   <2e-16 ***
## review_scores_cleanliness     4.180e+01  1.864e+00   22.43   <2e-16 ***
## review_scores_communication -2.007e+01  2.151e+00   -9.33   <2e-16 ***
## review_scores_location        6.838e+01  1.926e+00   35.49   <2e-16 ***
## review_scores_value          -3.984e+01  2.132e+00  -18.69   <2e-16 ***
## reviews_per_month            -1.228e+01  3.000e-01  -40.92   <2e-16 ***
## host_since_num               -8.877e-03  4.677e-04  -18.98   <2e-16 ***
## calendar_last_scraped_num    -1.666e-01  7.034e-03  -23.68   <2e-16 ***
```

```
## room_type_private_room      -2.843e+01  1.390e+00  -20.45   <2e-16 ***
## room_type_shared_room       -9.113e+01  5.489e+00  -16.60   <2e-16 ***
## citypacific_grove            1.034e+02  9.563e+00   10.81   <2e-16 ***
## citysan_francisco            3.474e+01  1.677e+00   20.71   <2e-16 ***
## citysanta_cruz_county        6.046e+01  3.470e+00   17.42   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 191.7 on 129104 degrees of freedom
## Multiple R-squared:   0.39,  Adjusted R-squared:  0.3899
## F-statistic:  5503 on 15 and 129104 DF,  p-value: < 2.2e-16
```

```
# Extract residuals (errors) from the training data
rk_train$residuals <- residuals(base_lmod)

# FIXED: Swapped out broken glmnet syntax matrix logic for a clean native LM prediction call
rk_train$y_pred <- as.numeric(predict(base_lmod, newdata = rk_train1[setdiff(names(rk_train1), "price")])
```

Residual check for normality

```
pdf("Residual_Normality_Diagnostics.RK.pdf", width = 10, height = 8)

# Set up a 2x2 grid framework for side-by-side visual analysis
par(mfrow = c(1, 2))

# Extract baseline training residuals (used as the input for Kriging)
rk_res <- rk_train$y_pred

# Plot 1: Histogram of Baseline RF Residuals
hist(rk_res, col = "skyblue", main = "Histogram: Baseline OLS Residuals",
     xlab = "Residual Error ($)", border = "white")
abline(v = mean(rk_res, na.rm=TRUE), col = "red", lwd = 2, lty = 2)

# Plot 2: Normal Q-Q Plot of Baseline RF Residuals
qqnorm(rk_res, main = "Q-Q Plot: Baseline RF Residuals", col = rgb(0,0,1, alpha=0.15), pch=19)
qqline(rk_res, col = "red", lwd = 2)



# Reset plotting layout parameters
par(mfrow = c(1, 1))
dev.off()
```

```
## pdf
##   2
```

```
# Convert flat data to spatial coordinates using standard WGS84
spatial_train_geo <- st_as_sf(rk_train, coords = c("longitude", "latitude"), crs = 4326)
spatial_test_geo  <- st_as_sf(rk_test, coords = c("longitude", "latitude"), crs = 4326)

# 3310 ( crs for California) covers the whole state
```

```r
spatial_train_projected <- st_transform(spatial_train_geo, crs = 3310)
spatial_test_projected  <- st_transform(spatial_test_geo, crs = 3310)

# Jitter coordinate duplicates to avoid spatial distance errors (0-meter distance bugs)
spatial_train <- st_jitter(spatial_train_projected, amount = 1.0)
spatial_test  <- st_jitter(spatial_test_projected, amount = 1.0)

# Check training residuals
any(is.na(spatial_train$residuals))
```

```
## [1] FALSE
```

```r
# Missing value check
any(is.na(spatial_test$price))
```

```
## [1] FALSE
```

```r
sub_sample_size <- min(5000, nrow(spatial_train))
spatial_train_sub <- spatial_train[sample(1:nrow(spatial_train), sub_sample_size), ]

# Compute empirical variogram
emp_vgm <- variogram(residuals ~ 1, data = spatial_train_sub)

# Find total sample variance
init_variance <- var(spatial_train_sub$residuals, na.rm = TRUE)

# Distributed variance stably between psill (70%) and nugget (30%)
init_vgm <- vgm(
  psill  = init_variance * 0.7,
  model  = "Sph",
  range  = 5000,
  nugget = init_variance * 0.3
)

# automated curve fitting
fitted_vgm <- fit.variogram(emp_vgm, model = init_vgm)
```

```
## Warning in fit.variogram(emp_vgm, model = init_vgm): singular model in
## variogram fit
```

```r
print(fitted_vgm)
```

```
##   model    psill range
## 1   Nug 10762.12     0
## 2   Sph 25111.62  5000
```

```r
# 2.Variogram plot

pdf("RK_Variogram.pdf", width=6, height = 5 )
plot(
```

```
  emp_vgm,
  model = fitted_vgm,
  main  = list(label = "Fitted Semivariogram Model of Regression Price Residuals", cex = 1.2),
  xlab  = "Distance (metres)",
  ylab  = "Semivariance"
)

dev.off()
```

```
## pdf
##   2
```

LOCAL SEARCH NEIGHBOURHOOD KRIGING ON TEST SET

```
rk_test$lm_pred <- predict(base_lmod, newdata = rk_test1[setdiff(names(rk_test1), "price") ])
actual_predictions <- rk_test$lm_pred
actual_test_prices <- rk_test$price

# Store baseline metrics in the tracking structures
rk_test$lmod_pred   <- actual_predictions
spatial_test$y_pred <- actual_predictions

# Verify calculation arrays
any(is.na(actual_predictions))
```

```
## [1] FALSE
```

```
summary(actual_predictions)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -590.1   111.3   185.4   219.3   295.7  1254.6
```

```
# Ordinary Kriging interpolation over isolated residual targets
test_res_kriged <- krige(
  formula   = residuals ~ 1,
  locations = spatial_train,
  newdata   = spatial_test,
  model     = fitted_vgm,
  nmax      = 50
)
```

```
## [using ordinary kriging]
```

```
# Extract and clean interpolated spatial correction scalars
kriged_residual_values <- test_res_kriged$var1.pred
kriged_residual_values[is.na(kriged_residual_values)] <- 0

# Combine baseline OLS predictions with spatial kriged adjustments
spatial_test$final_kriged_pred <- actual_predictions + kriged_residual_values
```

```r
# Checks for data gaps
cat("Number of skipped test points remaining:", sum(is.na(spatial_test$final_kriged_pred)), "\n")
```

```
## Number of skipped test points remaining: 0
```

```r
head(test_res_kriged)
```

```
## Simple feature collection with 6 features and 2 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: 144875.5 ymin: -461396.3 xmax: 177485.7 ymax: -431665.9
## Projected CRS: NAD83 / California Albers
##    var1.pred var1.var                   geometry
## 1 -90.621477 12255.16 POINT (144875.5 -446355.4)
## 2 206.389684 11915.78 POINT (148610.4 -435822.6)
## 3 -61.139511 13257.92 POINT (177485.7 -461396.3)
## 4 -23.083585 11775.29 POINT (160445.7 -432088.7)
## 5  -8.062859 12039.23 POINT (151585.8 -446494.4)
## 6  -5.092958 11587.47 POINT (154969.2 -431665.9)
```

```r
kriged_values <- as.numeric(test_res_kriged$var1.pred)
kriged_values[is.na(kriged_values)] <- 0

final_test_predictions <- actual_predictions + kriged_values

rk_test$final_kriged_pred <- final_test_predictions
spatial_test$final_kriged_pred <- final_test_predictions

# Verify training sets configurations
eval <- data.frame(
  obs  = rk_train$price,
  pred = rk_train$y_pred
)

cat("--- OLS Training Baseline Evaluation Top Rows ---\n")
```

```
## --- OLS Training Baseline Evaluation Top Rows ---
```

```r
print(head(eval))
```

```
##   obs      pred
## 1 110  82.25980
## 2 125  92.24333
## 3 135 179.88118
## 4 604 328.99493
## 5  55  71.12398
## 6 314 755.32307
```

```r
# Combined Regression-Kriging tracking dataframe
eval_test <- data.frame(
  obs  = rk_test$price,
  pred = spatial_test$final_kriged_pred
)

cat("\n--- Combined Regression-Kriging Evaluation Top Rows ---\n")
```

```
##
## --- Combined Regression-Kriging Evaluation Top Rows ---
```

```r
print(head(eval_test))
```

```
##   obs      pred
## 1  55 -51.02853
## 2 310 589.48856
## 3  65  77.80994
## 4 150 146.17753
## 5 124 151.22188
## 6 296 174.19307
```

```r
coords_test <- st_coordinates(spatial_test)
spatial_test$Easting  <- coords_test[, 1]
spatial_test$Northing <- coords_test[, 2]

# Open PDF device to capture the spatial distribution plot
pdf("spatial_regression_kriging_heatmap.pdf", width = 7, height = 5)
ggplot(data = spatial_test) +
  stat_summary_hex(
    aes(x = Easting, y = Northing, z = final_kriged_pred),
    fun = function(x) mean(x, na.rm = TRUE),
    bins = 40
  ) +
  scale_fill_viridis_c(option = "plasma", name = "Avg Price ($)") +
  theme_minimal() +
  labs(
    title = "Spatial Heatmap: Predicted Airbnb Prices",
    subtitle = "Aggregated Regional Price Trends (California - Regression Kriging)",
    x = "Easting (metres)",
    y = "Northing (metres)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right"
  )
dev.off()
```

```
## pdf
##   2
```

```
rk_test$spatial_error_pred <- kriged_values
rk_test$rk_final_pred <- rk_test$lm_pred + rk_test$spatial_error_pred

rk_res <- rk_test$rk_final_pred - rk_test$price
```

```
evaluate_and_save(rk_test$lmod_pred, rk_test$price, "Baseline Linear Model","diagnostic_baseline_lmod.p
```

```
## --- Baseline Linear Model Metrics ---
## MSE:        34821.06
## RMSE:       186.6
## MAE:        100.35
## R-squared:  0.3912
```

```
evaluate_and_save(rk_test$final_kriged_pred, rk_test$price, "Combined Regression Kriging", "diagnostic_
```

```
## --- Combined Regression Kriging Metrics ---
## MSE:        21423.35
## RMSE:       146.37
## MAE:        84.21
## R-squared:  0.6254
```

```
rk_rmse <- sqrt(mean((rk_test$price - rk_test$rk_final_pred)^2, na.rm = TRUE))
print(paste("Regression Kriging Test RMSE: $", round(rk_rmse, 2)))
```

```
## [1] "Regression Kriging Test RMSE: $ 146.37"
```

```
pdf("RK_Error.pdf", width=6, height=5)
ggplot(data = spatial_test) +
  geom_sf(aes(color = rk_res), size = 1.2, alpha = 0.7) +
  # Diverging scale: Red means underpriced, Blue means overpriced, Green is good.
  scale_color_gradient2(
    low = "blue", mid = "green", high = "red",
    midpoint = 0, name = "Prediction Error ($)"
  ) +
  theme_minimal() +
  labs(
    title = "Spatial Error Analysis: Where is the (RK) Model Wrong?",
    subtitle = "Red = Predicted Too Low | Blue = Predicted Too High",
    x = "Easting (metres)", y = "Northing (metres)"
  ) +
  theme(plot.title = element_text(face = "bold", size = 14))
dev.off()
```

```
## pdf
##   2
```